



HAWASSA UNIVERSITY

INSTITUTE OF TECHNOLOGY

FACULTY OF ELECTRICAL ENGINEERING

DEPARTMENT OF ELECTRICAL AND

COMPUTER ENGINEERING
Computer Programming
Course

Chapter 1: Introduction

- Computer and its uses
- Capabilities and limitations of computers
- Classification of computers
- Types of computers
- History of computers
- Basic operation of computers
- Organization of computer

WHAT IS a COMPUTER



Computer

- ✓ A Computer is an electronic device that accepts data, performs computations, and makes logical decisions according to instructions that have been given to it; then produces meaningful information in a form that is useful to the user.
- ✓ An electronic device that can perform activities that involve
 - ❖ Mathematical
 - ❖ Logical and
 - ❖ Graphical manipulations.

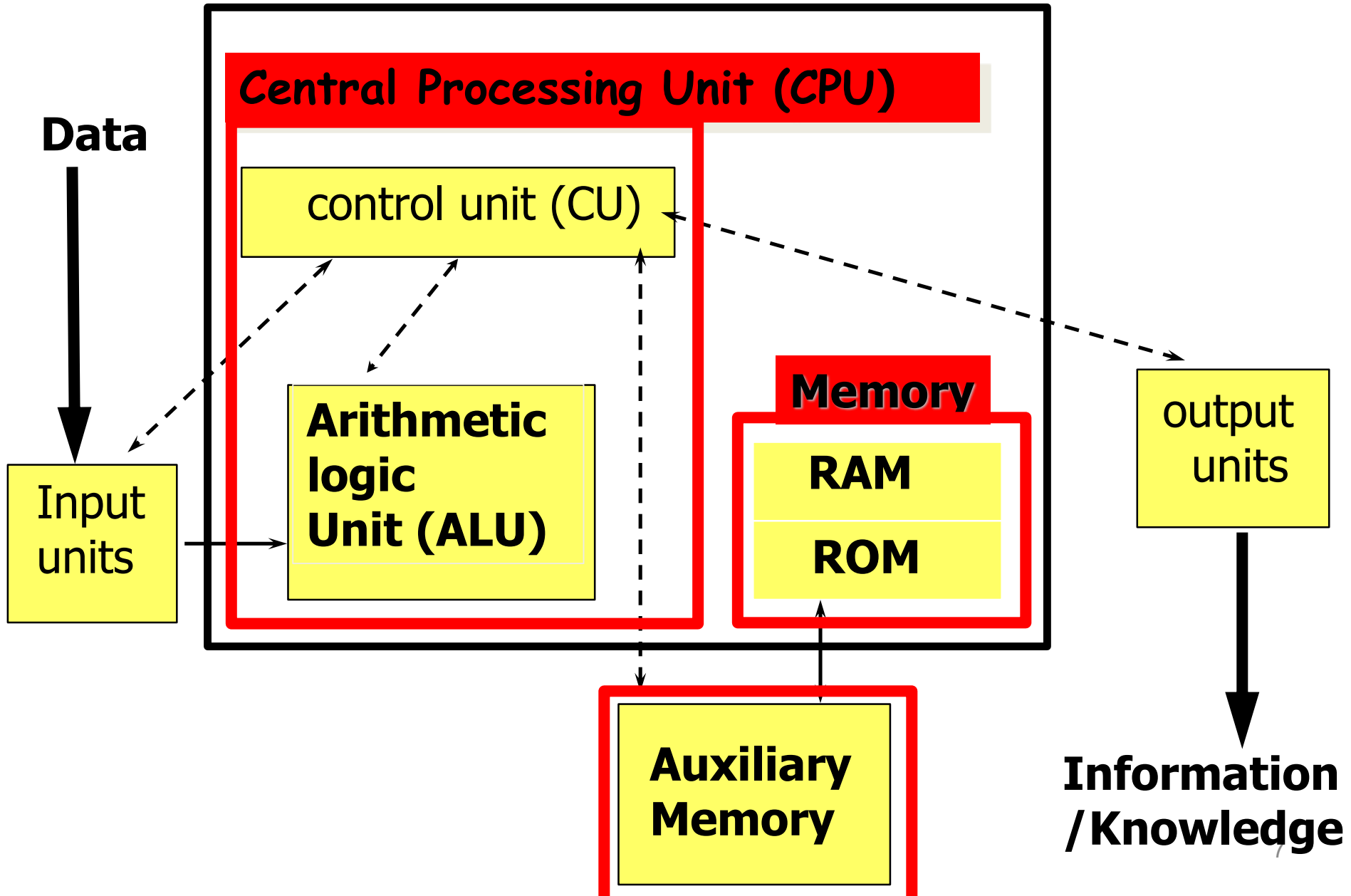
Cont....

- ✓ It performs the following three operations in sequence.
 1. It receives data & instructions from the input device.
 2. Processes the data as per instructions.
 3. Provides the result (output) in a desired form.
- ✓ **Data** : It is the collection of raw facts, figures & symbols.
Ex : Names of students and their marks in different subjects listed in random order.
- ✓ **Information** : It is the data that is processed & presented in an organized manner.
Ex : When the names of students are arranged in alphabetical order, total and average marks are calculated & presented in a tabular form, it is information.

To compute = To calculate

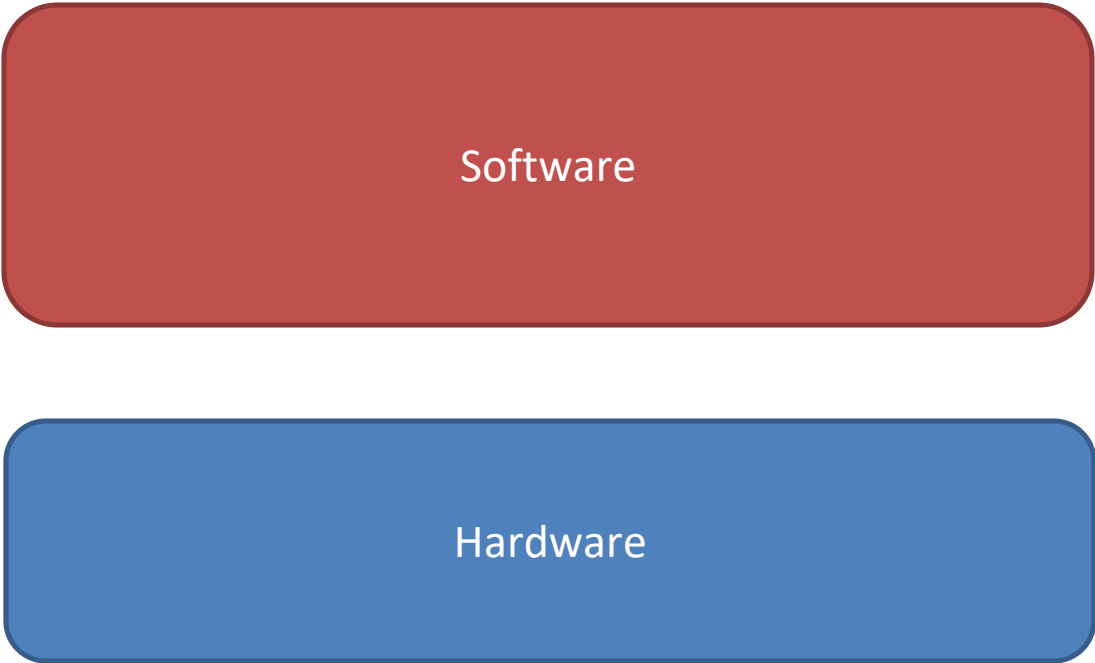
- ✓ Generally, the term is used to describe a collection of devices that function together as a system.
- ✓ A computer contains several major subsystems such as the **Central Processing Unit (CPU)**, **memory**, and **peripheral device** controllers. These components all plug into a "Bus". The bus is essentially a communications highway; all the other components work together by transferring data over the bus.

Components of a Computer System



Overview of Computer system

Basically the computer system is categorized into two components:



The diagram consists of two vertically stacked rounded rectangular boxes. The top box is red and contains the word 'Software'. The bottom box is blue and contains the word 'Hardware'.

Software

Hardware

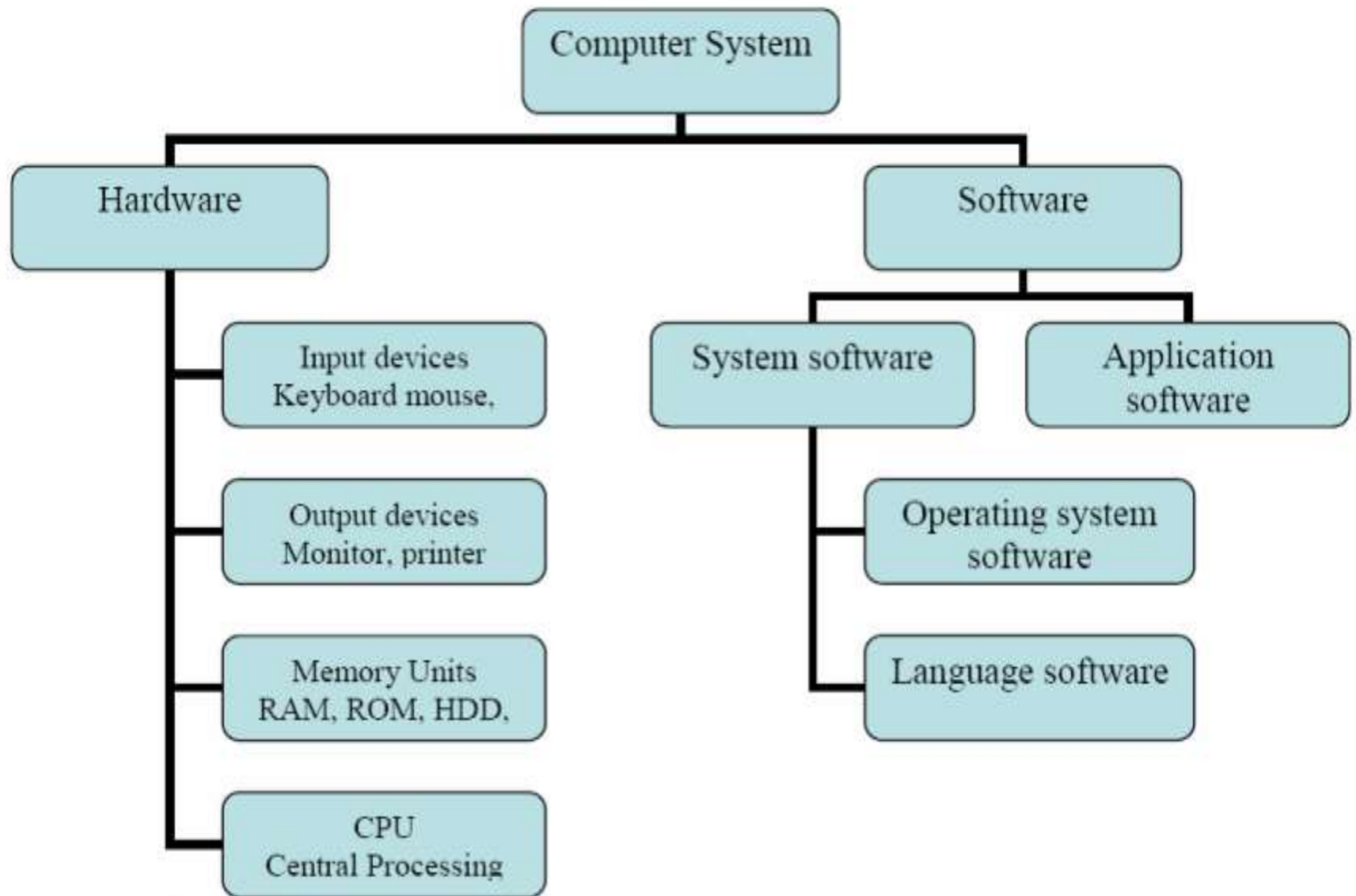


Figure 2.1 Components of Computer System

Computer Systems - Hardware

1. Hardware

- All physical component of a computer

- Contain major components:

- CPU

- Run programs

- Fetch->Decode>execute

- Control Unit & ALU

- RAM

- CPU can randomly access data

- Volatile & Divided in byte storage unit

- Secondary Storage

- Save data for long period, eg: HDD, CD, DVD, USB Flash...

- I/O Devices

- **[INPUT]** Collect Info → Send PC & **[OUTPUT]** PC → Present Data



The Components of a Computer

What are common computer hardware components?



The Components of a Computer

What is an input device?

- Any hardware component that allows a user to enter data and instructions into a computer



The Components of a Computer

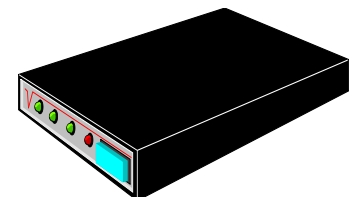
What is an output device?

- Any hardware component that can convey information to a user



The Physical Components

- ✓ Input devices
- ✓ Processing
- ✓ Output devices
- ✓ Storage devices
- ✓ Communications devices

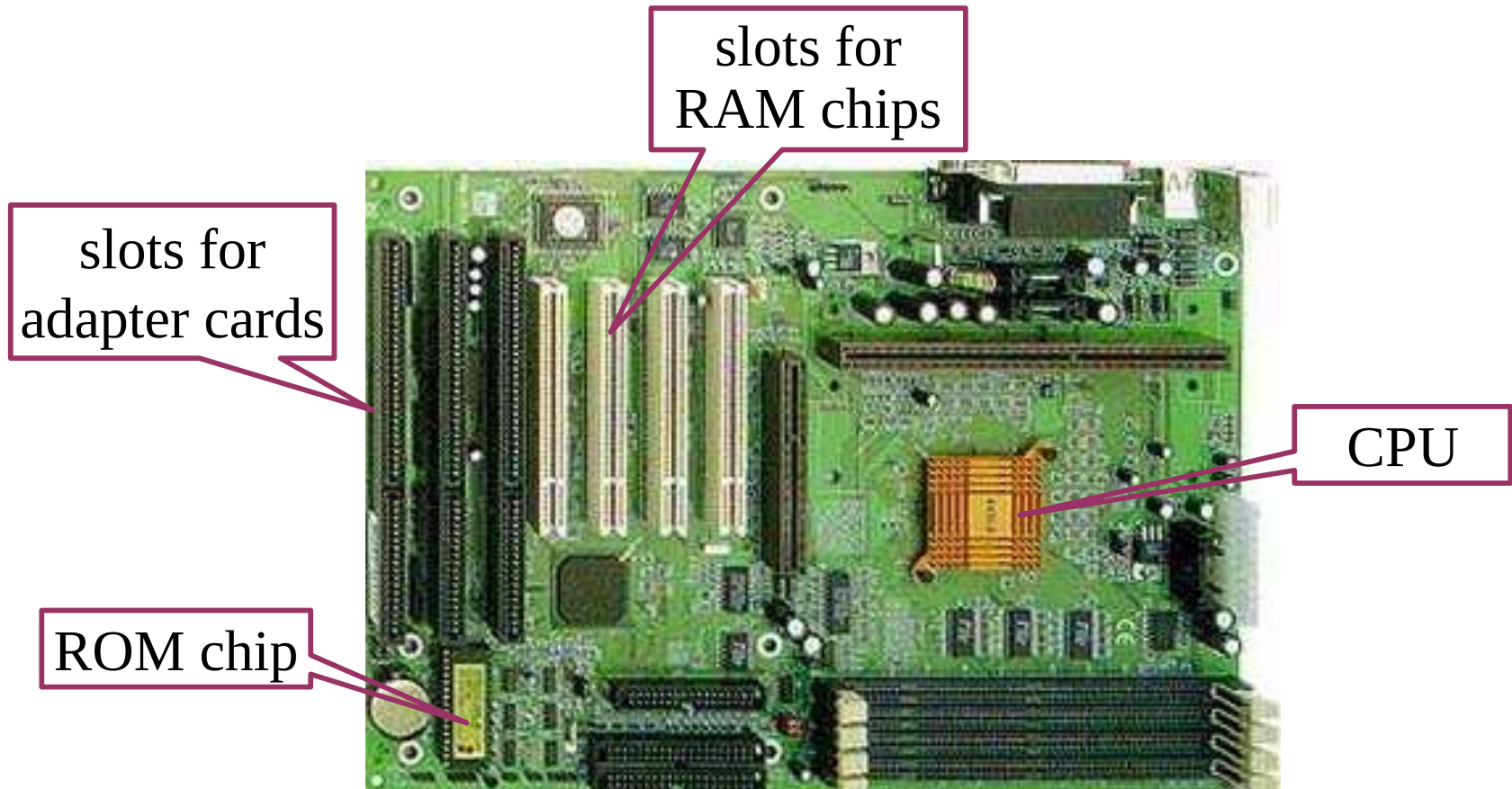


Common PC Port Types

- ✓ Serial - bits flow in a single line
- ✓ Parallel - several bits flow at once
- ✓ SCSI - can connect up to 8 devices, fast data transfer
- ✓ USB - can connect many devices, has *hot swapping* capability
- ✓ FireWire - like USB, higher speed & more \$
- ✓ IrDA - infrared, line of sight

Motherboard

✓ Printed circuit board with connectors

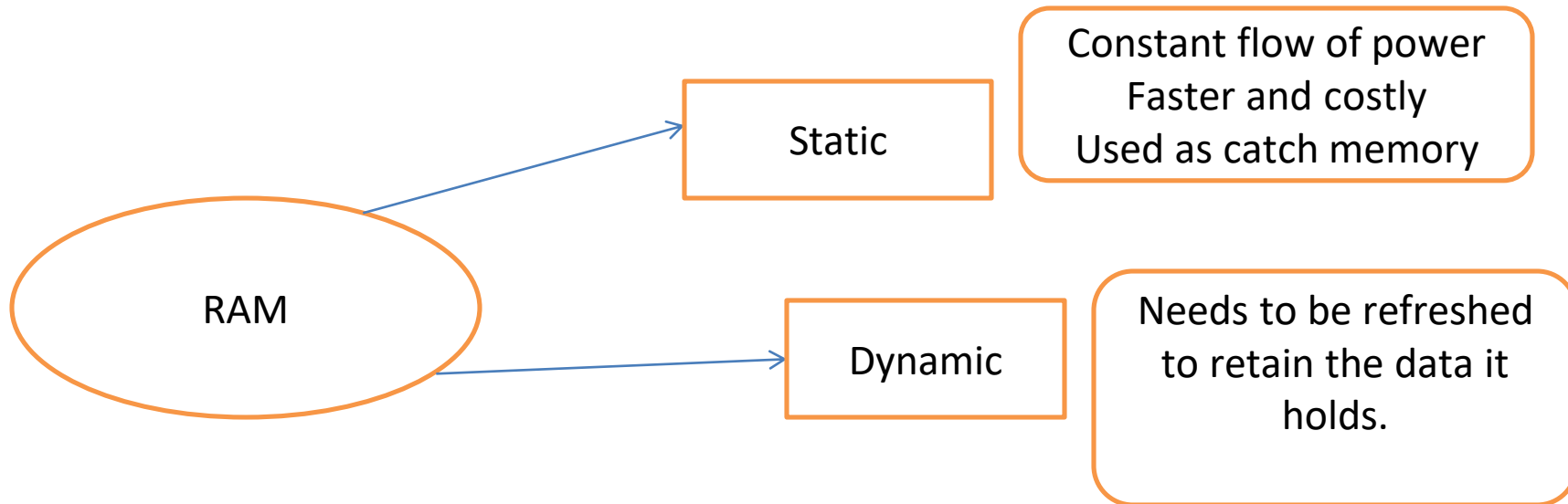


CPU

- ✓ A device containing millions of electronic parts
- ✓ The brain of the computer
 - Does all the fundamental computation
 - Controls all other components
- ✓ Performs computations by executing instructions in sequence at discrete time intervals

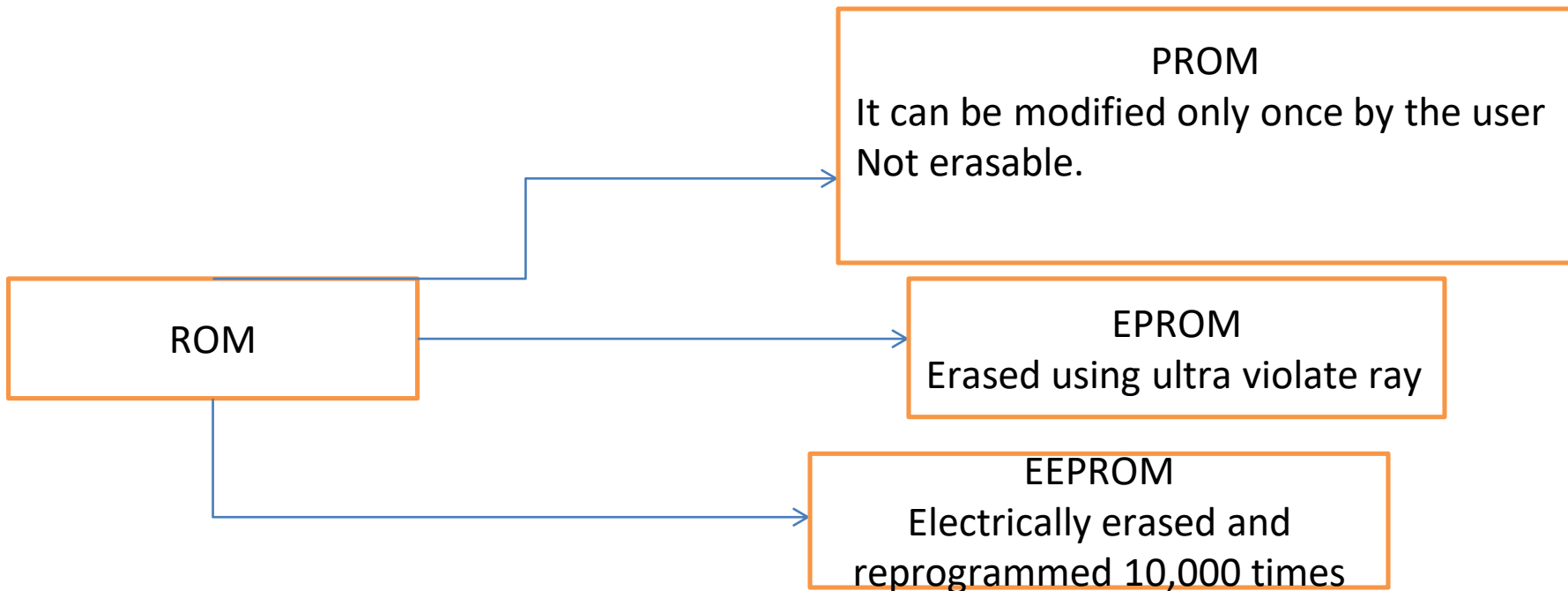
Memory - RAM

- ✓ Closely connected to the CPU, volatile
- ✓ Stores part of operating system
- ✓ High capacity.
- ✓ Fastest and costliest memory.
- ✓ Stores part of the application programs
- ✓ Stores your files as you work on them
- ✓ Read and write memory.



ROM

- ✓ Non volatile
- ✓ Capacity, Smaller than RAM
- ✓ Slower and cheaper than RAM
- ✓ Data can only be read by cpu
- ✓ Stores instruction that computer requires during bootstrapping.



RAM

- It is read -write memory
- Used to store the data that has to be currently presented by CPU.
- Volatile
- Data can be modified
- 64MB to 16GB
- Expensive

ROM

- ❖ It is read only memory
- ❖ It stores the instructions required during bootstrap of the computer.
- ❖ Non- volatile.
- ❖ Data can not be modified
- ❖ Comparatively smaller than RAM
- ❖ Cheaper

Secondary Storage - CD/DVD

- ✓ *-R - recordable burn once

- ✓ *-RW - rewritable
erase and burn again

- ✓ 650 MB for a CD

- ✓ 4.7 GB for a DVD

- ✓ 8.5 GB for Dual Layer
DVDs

- ✓ In-expensive



Flash memory - USB stick

Flash memory is a non-volatile computer memory that can be electrically erased and reprogrammed.

Easy to use and fairly inexpensive.



Computer Systems - Software

2. Software

- ❑ Control/Govern of computer
- ❑ Two Categories:

1. System Software

- ❑ Control/Manage basic programs

1. **Operating System(OS):** control internal operations of HW, Manage I/O devices, help other programs to run

Example: Windows 10, 8, Vista, Xp..., MacOS, Android, iOS, Ubuntu, Linux, ChromeOS...

2. **Utility Programs:** special task operations

Example: File Compressor, Virus Scanner, ...

3. **Software Development Tools:** tools used by programmers to develop software products

Example: IDE's, Compilers, Linkers,...

2. Application Software

- ❑ Help users to use computers for a certain task
- ❑ Operate at the top of OS
- ❑ Example: MS Word, Music Players, Browsers, ...

Operating System

Application Software

Cont....

- ✓ **Program** : Set of instructions that enables a computer to perform a given task.
- ✓ **Advantages of computers**
 1. High speed
 2. Accuracy
 3. Storage
 4. Automation
 5. Diligence
 6. Versatility
 7. Cost effectiveness
- ✓ **Limitations of computers** :
 1. Computers need clear & complete instructions to perform a task accurately. If the instructions are not clear & complete, the computer will not produce the required result.
 2. Computers cannot think.
 3. Computers cannot learn by experience.

Classification based on hardware design and data handling

✓ **Analog**

Voltmeter

Thermometer

Speedometer

Gas oil pump

They have limited accuracy.

✓ **Digital**

Abacus

Desk. and pocket computer

General purpose computer

Hybrid

Hospital ICU

Classification based on purpose

Special purpose

- ✓ public telephone box
- ✓ traffic control system
- ✓ ticket machines
- ✓ pocket calculators etc.
- ✓ counters

General purpose

- ✓ Micro computer
- ✓ Mini computer
- ✓ Super computers

Classification based on price, capacity, size and performance

Micro computer

Mini computer

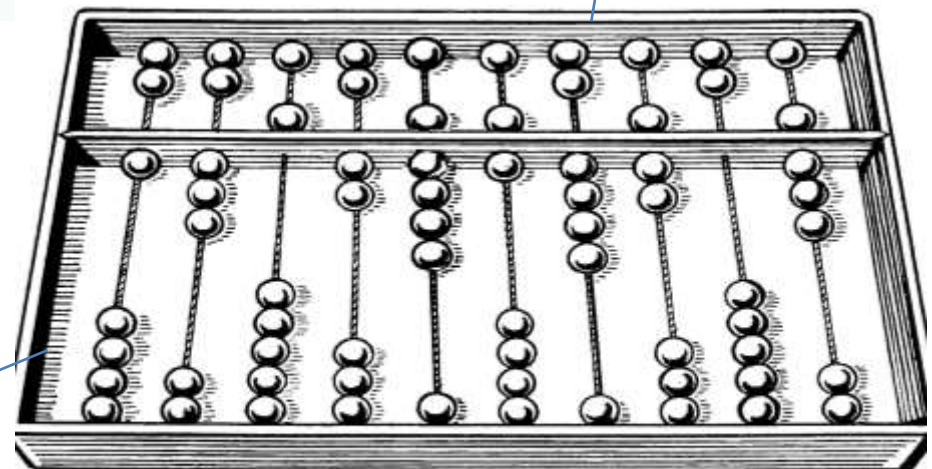
Mainframe computer



Tools to aid with calculation – Coffee Beans?

Do you use or have used any of these?

earth beats



Haven beats

1 3 5 2 9 6 4 7 0 8

Tools to aid with calculation – The Abacus

500 years ago

Has beads and roads



Tools to aid with calculation – The Slide Rule

Do you use or have used any of these?



Pascaline

1642 Blaise Pascal

Has wheels and gears (taxi meters)

First mechanical calculator in the world.

Charles Babbage(1833)

British mathematician

Who introduced idea of storing and reading the information before processing.



Tabulating Machine

Harman Hollerith (1890) usa statistician

Can read data

Processing data

Operates based on IPO cycle.

Can read both letters and numbers



ENIAC & UNIVAC

ENIAC (electronic numerical integrator and calculator) uses decimal instated of Binary. weighting 30 tones, 15,000 square feet of floor space over 18,000 vacuum tubes. Capable of adding 5,000 additions.

UNIVAC (universal Automatic computer) first commercially used electronic computer work with both numerical and textual data.

S.No	Generation & Description
1	First Generation  The period of first generation: 1946-1959. Vacuum tube based.
2	Second Generation  The period of second generation: 1959-1965. Transistor based.
3	Third Generation  The period of third generation: 1965-1971. Integrated Circuit based.
4	Fourth Generation  The period of fourth generation: 1971-1980. VLSI microprocessor based.
5	Fifth Generation  The period of fifth generation: 1980-onwards. ULSI microprocessor based.

Generations of computer

➤ 1st generation of computer (1941-1956) 2nd world war

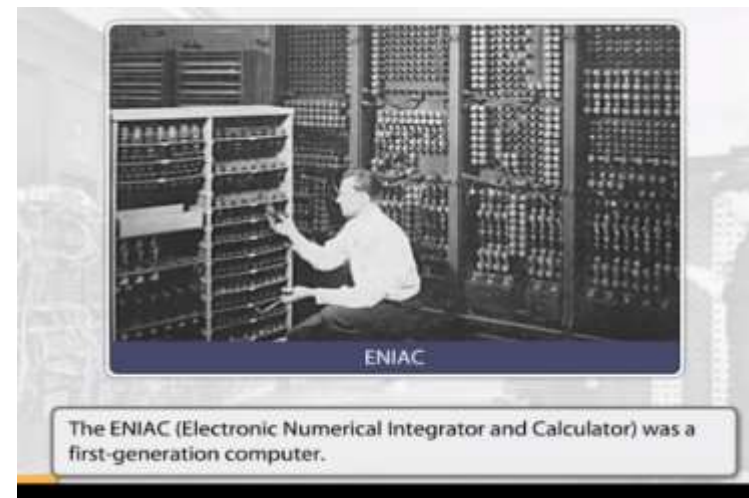
- Uses vacuum tube consists of glass bulb and wire. The wire is used to carry data in the form of electronic signals.

Machine language is used



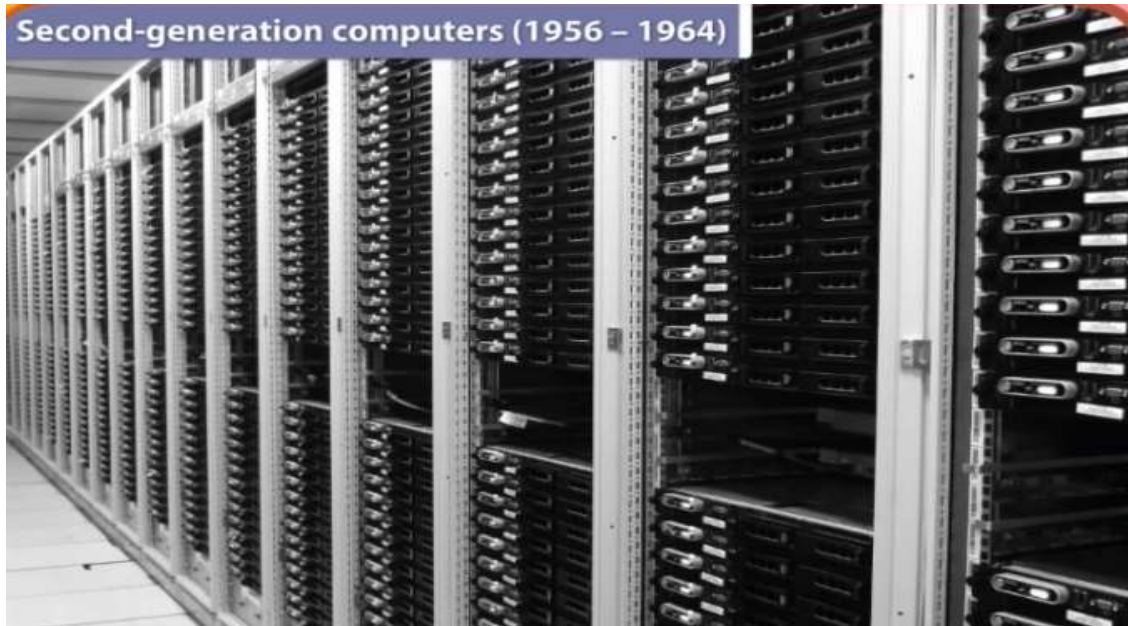
ENIAC
UNIVAC

- Very large ,expensive and required huge amounts of electricity.



Cont..

- 2nd generation of computer (1956-1963)



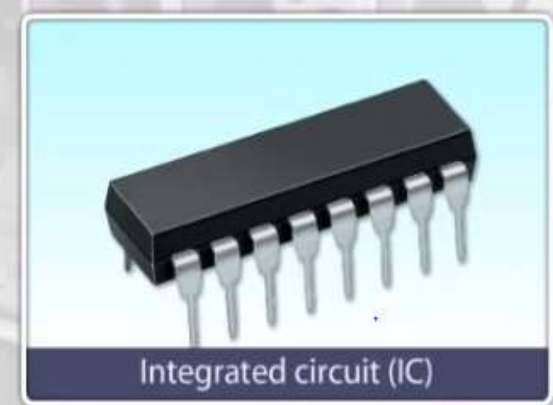
Uses transistors
instated of vacuum
tubes.

- These computer is faster than 1st G, and cheaper
- Used in atomic energy industry.

IBM 700
IBM 650
ATLAS

Cont..

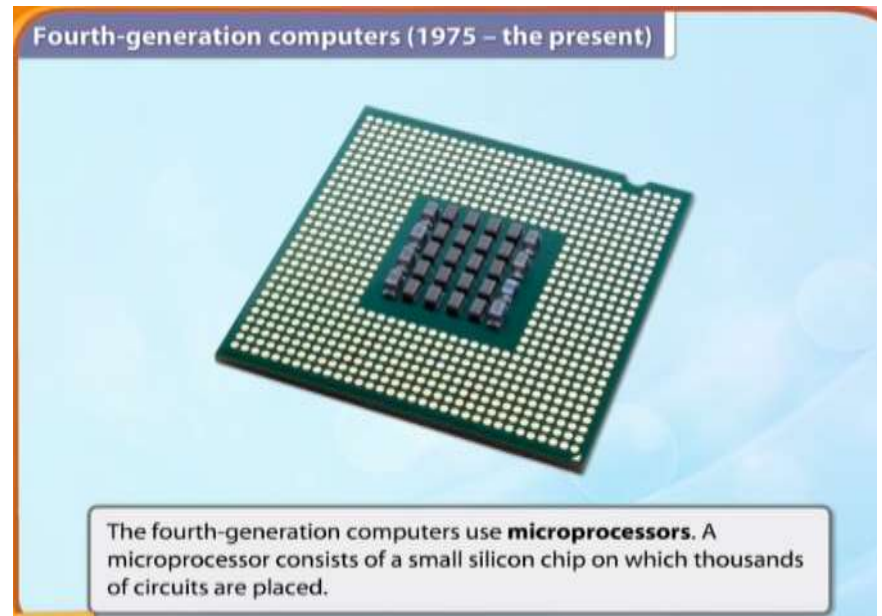
- 3rd generation of computer (1964-1971)



Can run different programs at a time
IBM 1130
UNIVAC 1107
Uses high level programming language.

IC made computers faster, cheaper and smaller than the second G.
They use less electricity and made fewer mistakes.

4th G



IBM 4341

DEC 10

STAR 1000

Uses high level programming language

5th G.

Fifth-generation computers (the future)



The fifth-generation computers are the most advanced computers.



Robot

PC (Personal Computer)



Workstation



Minicomputer



Mainframe



Supercomputer

Business



Banking



Insurance



Marketing



Insurance

Education



Healthcare



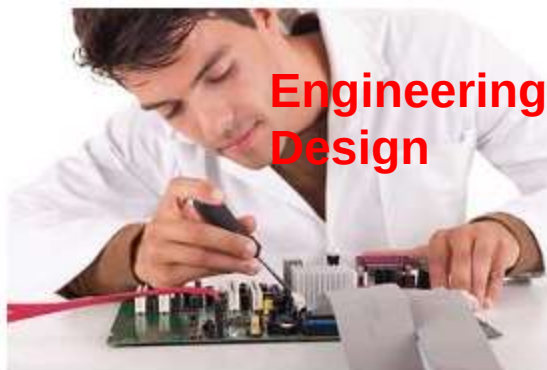
Communication



Military



Engineering Design



Government



PROGRAMMING LANGUAGES

- ❖ A **programming language** is an artificial language that can be used to control the behavior of a machine, particularly a computer.
- ❖ Programming languages, like human languages, are defined through the use of syntactic and semantic rules, to determine structure and meaning respectively.
- ❖ Programming languages differ from natural languages in that natural languages are only used for interaction between people, while programming languages also allow humans to communicate instructions to machines. Examples: BASIC, FORTRAN, c, c++ etc.

NUMBER REPRESENTATION IN COMPUTERS

- The representation of numbers in computers is usually not based on the decimal system, and therefore it is necessary to understand how to convert between different systems of representation.

System	Base	Symbols	Used by humans?	Used in computers?
Decimal	10	0, 1, ... 9	Yes	No
Binary	2	0, 1	No	Yes
Octal	8	0, 1, ... 7	No	No
Hexa-decimal	16	0, 1, ... 9, A, B, ... F	No	No

End of Chapter 1



HAWASSA UNIVERSITY

INSTITUTE OF TECHNOLOGY

FACULTY OF ELECTRICAL ENGINEERING

**DEPARTMENT OF ELECTRICAL AND COMPUTER
ENGINEERING**

**Chapter Two:
C++ Programming Basics**

By: Dejene H.

Contents

- ✓ Programs and Programming Languages
- ✓ The programming process
 - (Compile Link and Run process)
 - Source code , Object code, and Executable code
 - The integrated development environment (IDE)
- ✓ Program Design
 - Algorithms
 - Flowcharting (Starting out with C++, appendix O)
- ✓ A brief look at the program development cycle

The Basics

- ✓ **Program:** a list of *instructions* that tells the computer how to solve a problem or perform a task.
- ✓ **Programming:** the act of defining these instructions.
- ✓ **Programmer:** the person doing the defining.
- ✓ **User:** the person who ultimately makes use of the program. (Examples??)

Remember

- ✓ Computers are digital devices that simply do what they are told.
- ✓ We must speak their language to tell them what to do.
- ✓ Does this mean, we should speak binary? (0's and 1's)??



Programming Languages



Very High Level
Languages
(SQL, Prolog, ...)

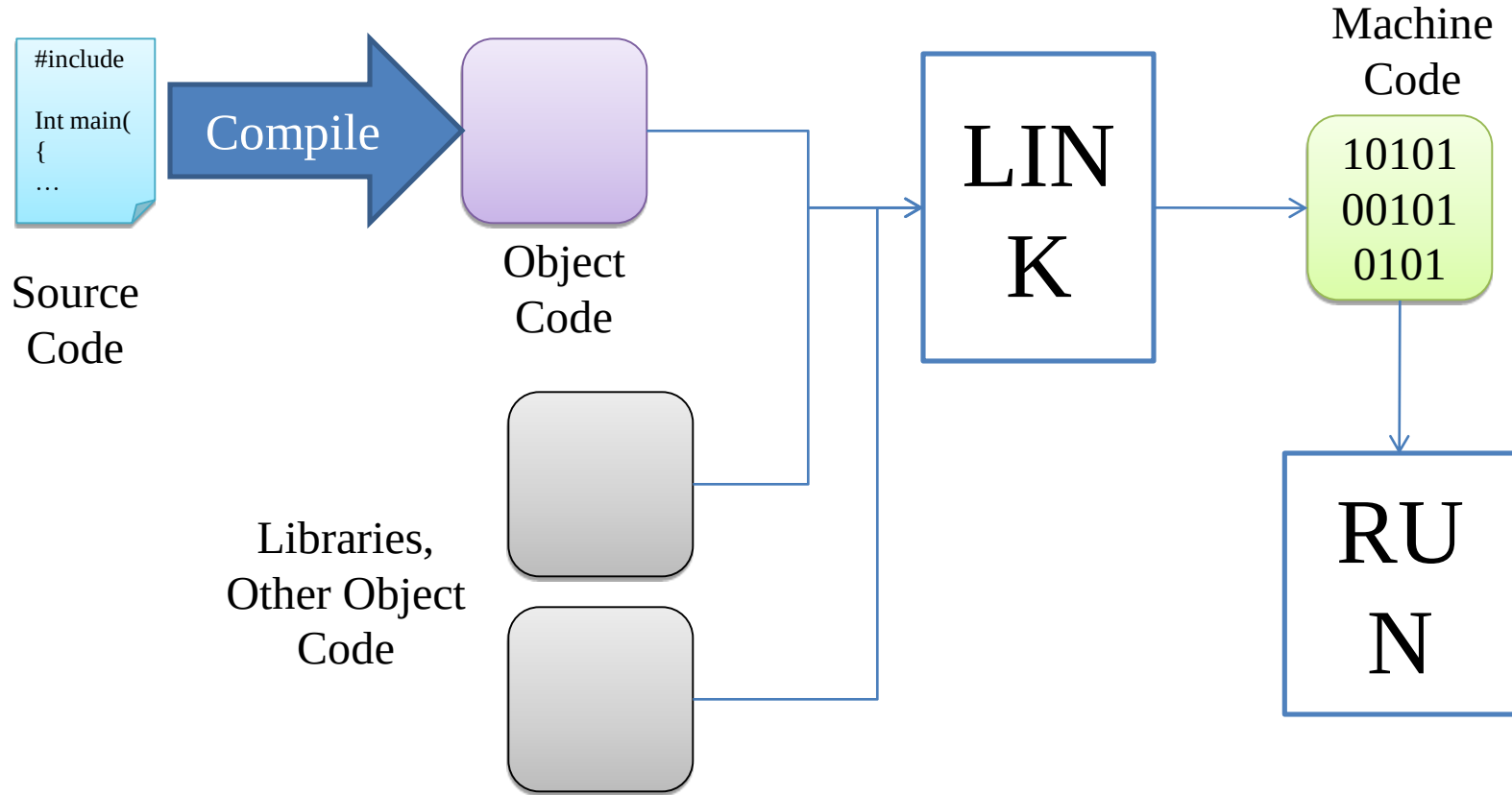
High Level
Languages
(C++)

Low Level
Languages
(Assembly, Machine Code)

Features of High Level Languages

- ✓ Designed to make programming easier
- ✓ Emphasis on logic
- ✓ Aids in the design process
- ✓ Independence (relatively) from the Operating System and Hardware
- ✓ One HLL instruction gets translated to multiple machine level instructions

The Programming Process



Programming Process

- ✓ IDE is a programming environment that has been packaged as an application program, typically consisting of:-Source code editor, a compiler, a debugger, and a graphical user interface (GUI) builder
- ✓ A **source code editor** is a text editor program designed specifically for editing source code of computer programs by programmers.
- ✓ A **compiler** is a computer program (or set of programs) that transforms source code written in a computer language (the source language) into another computer language (the target language, often having a binary form known as object code).

Cont...

- ✓ **Linker or link editor** is a computer program that takes one or more object files generated by a compiler and combines them into a single executable program.
- ✓ A **debugger** or debugging tool is a computer program that is used to test and debug other programs (the "target" program).
- ✓ It finds the errors in a given program whether it meets the target or not.
- ✓ Writing codes, =>Compiling written source code =>Running compiled file. We will use dev C++ software

Problem Solving Techniques

- ✓ Computer solves varieties of problems that can be expressed in a finite number of steps leading to a precisely defined goal by writing different programs. A program is not needed only to solve a problem but also it should be reliable, (maintainable) portable and efficient.
- ✓ In computer programming two facts are given more weight:
 - ❑ The first part focuses on defining the problem and logical procedures to follow in solving it.
 - ❑ The second introduces the means by which programmers communicate those procedures to the computer system so that it can be executed.

CONT...

- ✓ Before a program is written, the programmer must clearly understand what data are to be used, the desired result, and the procedure to be used to produce the result.
- ✓ The procedure, or solution, selected is referred to as an **algorithm**.
- ✓ An **algorithm** is defined as a step-by-step sequence of instructions that must terminate and describe how the data is to be processed to produce the desired outputs.

The Blueprint

- ✓ You can think of an algorithm as the blueprint (plan) of your program.
- ✓ Before you begin writing a program, you first have to understand the problem and then develop a step-by-step solution (the algorithm).
- ✓ Developing an algorithm to solve a problem is usually more difficult than writing the code.

Characteristics of Algorithms

- ✓ An algorithm for a computer program must
 - Have precisely defined steps
 - Not be ambiguous
 - Terminate after a finite number of steps
- ✓ An algorithm can have
 - Sequence (Process)
 - Decision (Selection)
 - Repetition (Iteration/Looping)

Sequence

- ✓ Each task follows the previous one without altering the order of the tasks. Each task is executed once.

Decision

- ✓ Allows only one of two tasks to be carried out, depending on a controlling logical condition. The selected task is executed only once. If...then..., If... then... else...
- ✓ If today is Sunday then don't go to school
- ✓ If today is Saturday or today is Sunday then don't go to school, else go to school
- ✓ The outcome of a decision is either true or false.

Repetition

- ✓ Enables a task to be repeated until a controlling condition no longer holds.

Keep going to school until you graduate.

Repeat this example until the students understand and go to the next example when they do.

- ✓ What if it loops infinitely???

More Concepts - Variables

Input/output

✓ *Your algorithm usually needs a set of inputs from the user and may give outputs to the user.*

Variables

✓ -are a way of representing data in your programs. You can think of them as containers for a value.

CONT...

- ✓ There are three commonly used tools to help to document program logic (the algorithm). These are **flowcharts**, **Hierarchy chart**, and **Pseudocode**.

Expressing Algorithms

1. Hierarchy chart
2. Pseudocode (False code)
 - Uses much restricted vocabulary
 - Closer to a Programming Language
 - Uses variables and symbols
 - There are different types available
3. Flowchart
 - Graphical method
 - Uses various symbols for various actions
 - Very easy to draw
 - Very easy to read and understand

1.Pseudocode

- ✓ It can be written in two forms: High level or detailed level.

Example 1:Customer age and gender

Get customer age and gender

Decide whether customer is eligible

Open if eligible and close otherwise

How to write and run a C++ program.....Example 2

1. Using a text editor or IDE and write the program
2. Save it as a file (.cpp extension)
3. Fix any errors that you might have
4. Compile your program into object code.
5. If compilation is successful, continue to step 6. If it fails, go back to step 3.
6. Run the linker to generate executable code.
7. If linking is successful, continue to step 8. Otherwise, go back to step 3.
8. Run (execute) your program and test it.
9. If there are errors, go back to step 3. Otherwise finish.

Example 3: Pseudocode for a cup of tea ?



CONT...

1. Get tea-making equipment:

- Open cupboard
- Get cup
- Get teabag
- Get sugar
- Close cupboard
- Put teabag in cup
- Put sugar in cup



3. Fill kettle from tap:

- a. Place kettle under tap
- b. Turn on tap
- c. Turn off tap



4. Boil kettle:

- Plug kettle in
- Turn kettle on

5. Pour water into cup

6. Stear

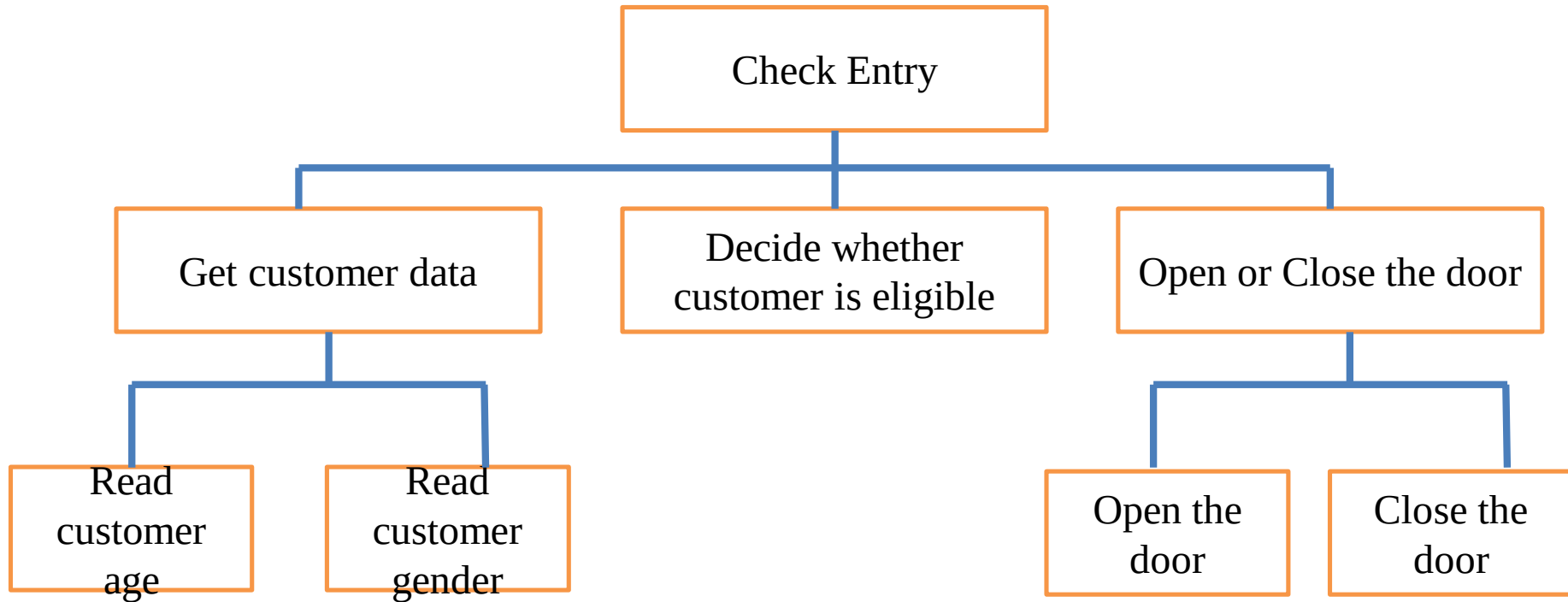
7. Drink



2.Hierarchy Chart

- ✓ It is graphical representation of design of a program
- ✓ It starts with overall task and divides it into smaller subtasks.
- ✓ So that it can be named "top-down design"

Hierarchy Chart

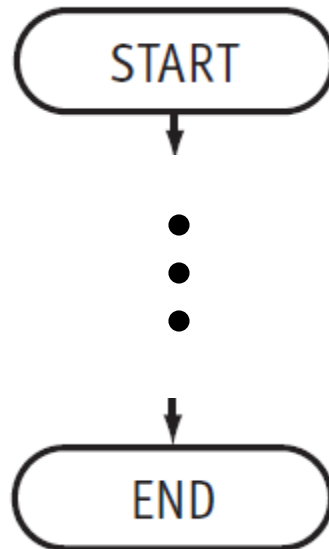


3.Flowchart Basics

- ✓ A flowchart represents an algorithm or process, showing the steps as various symbols and showing their order by connecting them with arrows.
- ✓ The basic symbols are used in flowcharts are discussed here

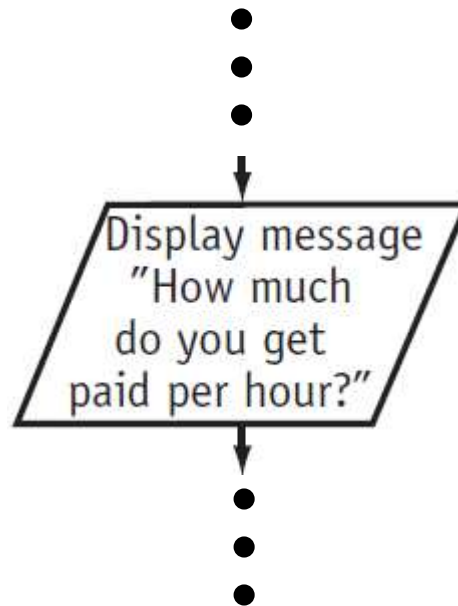
Terminal Points

- ✓ Rounded rectangles, or terminal points, indicate the starting and ending points of the flowchart.



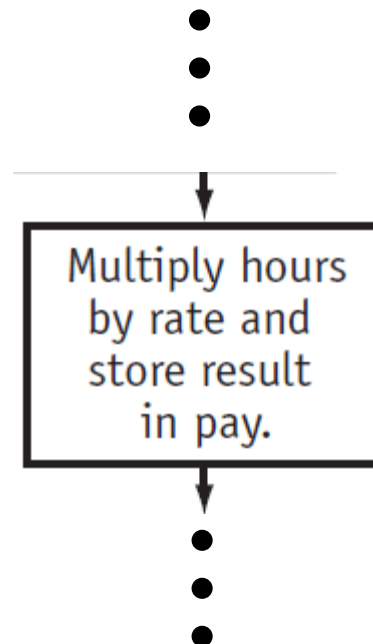
Input/output Operations

- ✓ Parallelograms designate input or output operations.



Processing

- ✓ A rectangle depicts a process such as a mathematical computation or a variable assignment.

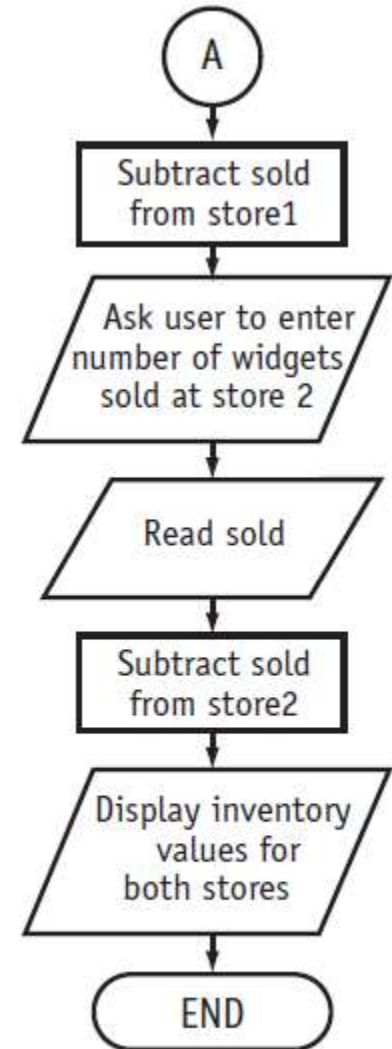
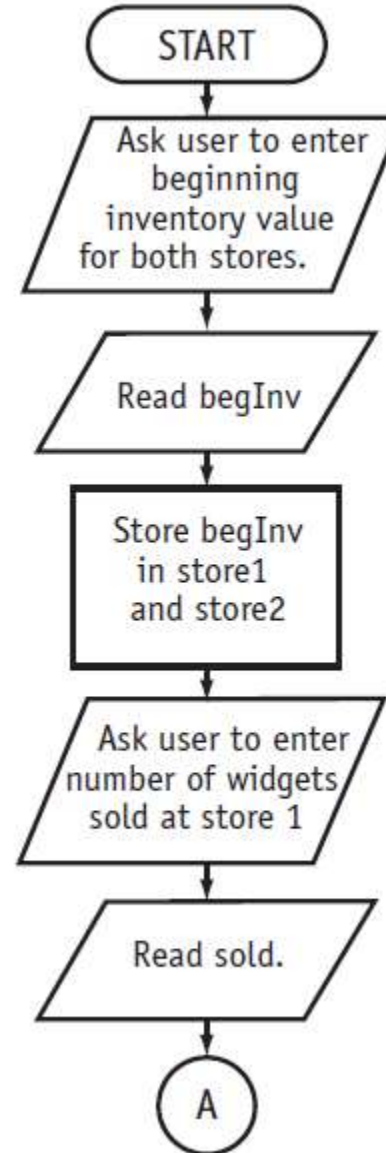
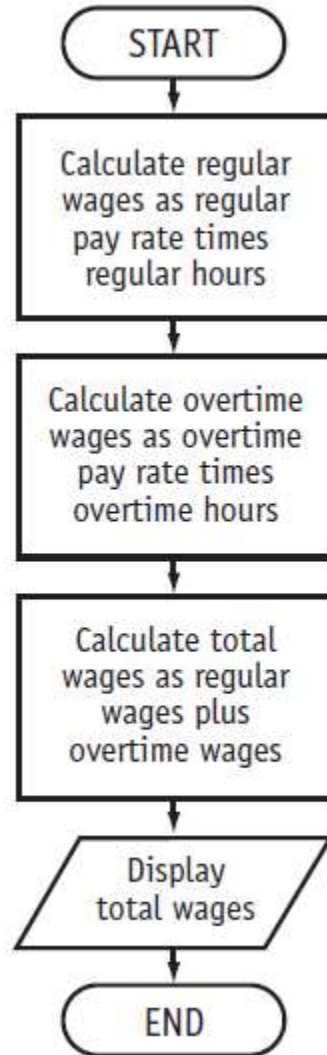
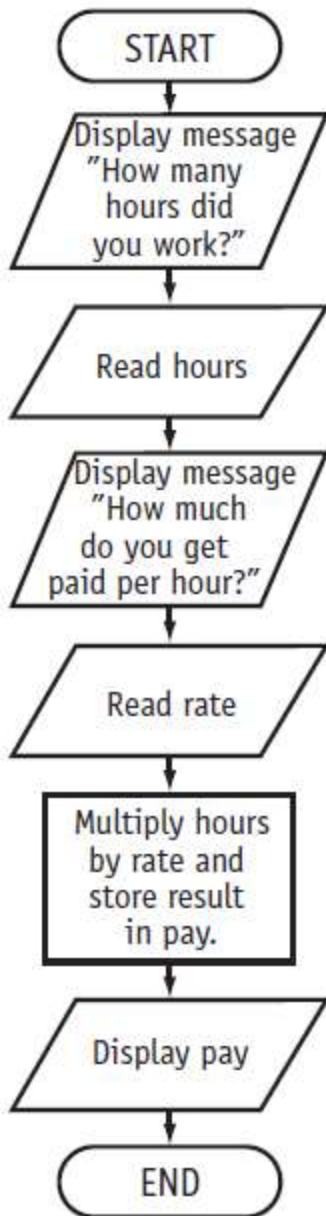


Connectors

- ✓ A connector symbol, which is a circle, is used to break flowchart into multiple parts.

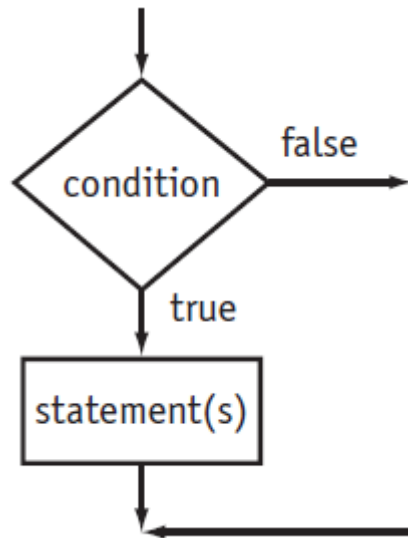


Example Flowcharts



The Decision Structure

- ✓ With decisions, the path to follow is determined by the outcome of a test. The diamond symbol is used to represent the test to be performed.



Basic Program Construction

A (Very) Simple C++ Program

```
1 // my first program in C++
2
3 #include <iostream>
4 using namespace std;
5
6 int main ()
7 {
8     cout << "Hello World!";
9     return 0;
10 }
```

Hello World!

Probably the best way to start learning a programming language is by writing a program. Therefore, the above shall be **our first program**.

Structure of a C++ Program

```
1 // my first program in C++
2
3 #include <iostream>
4 using namespace std;
5
6 int main ()
7 {
8     cout << "Hello World!";
9     return 0;
10 }
```

```
Hello World!
```

This is a ***comment line***. All lines beginning with two slash signs (//) are considered comments and do not have any effect on the behavior of the program. The programmer can use them **to include short explanations** or observations within the source code itself. In this case, the line is a brief description of what our program is.

Structure of a C++ Program

```
1 // my first program in C++  
2  
3 #include <iostream>  
4 using namespace std;  
5  
6 int main ()  
7 {  
8     cout << "Hello World!";  
9     return 0;  
10 }
```

Hello World!

Lines beginning with a hash sign (#) are **preprocessor directives**. They are not regular code lines with expressions but indications for the compiler's preprocessor. In this case the directive `#include<iostream>` tells the preprocessor **to include the iostream standard file**. This specific file (iostream) includes the declarations of the basic standard input-output library in C++, and it is included **because its functionality is going to be used later in the program**.

Structure of a C++ Program

```
1 // my first program in C++  
2  
3 #include <iostream>  
4 using namespace std;  
5  
6 int main ()  
7 {  
8     cout << "Hello World!";  
9     return 0;  
10 }
```

Hello World!

All the elements of the standard C++ library are declared within what is called a **namespace**, the namespace with the name ***std***. So in order to access its functionality we declare with this expression that we will be using these entities.

Structure of a C++ Program

```
1 // my first program in C++
2
3 #include <iostream>
4 using namespace std;
5
6 int main ()
7 {
8     cout << "Hello World!";
9     return 0;
10 }
```

Hello World!

This line corresponds to the beginning of the definition of the **main function**. The main function is the point by where all C++ programs start their execution, independently of its location within the source code. It is essential that all C++ programs have a *main* function. The word *main* is followed in the code by a pair of parentheses (**()**). Right after these parentheses we can find the body of the main function enclosed in braces (**{}**). What is contained within these braces is what the function does when it is executed.

Structure of a C++ Program

```
1 // my first program in C++
2
3 #include <iostream>
4 using namespace std;
5
6 int main ()
7 {
8     cout << "Hello World!";
9     return 0;
10 }
```

Hello World!

This line is a **C++ statement**. A statement is a simple or compound expression that can actually produce some effect. In fact, this statement performs the only action that generates a visible effect in our first program. A statement ends with a semicolon character (;). Note that *cout* is included within the *std* namespace of the *iostream* standard file and is used to display characters in the standard output stream like the computer monitor, for example.

Structure of a C++ Program

```
1 // my first program in C++
2
3 #include <iostream>
4 using namespace std;
5
6 int main ()
7 {
8     cout << "Hello World!";
9     return 0;
10 }
```

Hello World!

The **return** statement **causes the main function to finish**. `return` may be followed by a return code (in our example is followed by the return code with a value of zero). A return code of *0* for the *main* function is generally interpreted as the program worked as expected without any errors during its execution. This is the most usual way to end a C++ console program.

Structure of a C++ Program

```
1 // my first program in C++
2
3 #include <iostream>
4 using namespace std;
5
6 int main ()
7 {
8     cout << "Hello World!";
9     return 0;
10 }
```

Hello World!

You may have noticed **that not all the lines of this program perform actions when the code is executed**. There were lines containing only **comments** (those beginning by `//`). There were lines with **directives** for the compiler's preprocessor (those beginning by `#`). Then there were lines that began the declaration of a function (in this case, the **main** function) and, finally lines with statements (like the insertion into ***cout***), which were all included within the block delimited by the braces (`{}`) of the main function.

The program has been structured in different lines in order to be more readable, but in C++, we do not have strict rules on how to separate instructions in different lines. For example. instead of

```
int main ()  
{  
    cout << " Hello World!";  
    return 0;  
}
```

We could have written:

```
int main () { cout << "Hello World!"; return 0; }
```

All in just one line and this would have had exactly the same meaning as the previous code.

In C++, the separation between statements is specified with an ending **semicolon** (;) at the end of each one, so the separation in different code lines does not matter at all for this purpose.

We can write many statements per line or write a single statement that takes many code lines. The division of code in different lines serves only to make it more legible and schematic for the humans that may read it.

Let us add an additional instruction to our first program:

```
1 // my second program in C++
2
3 #include <iostream>
4
5 using namespace std;
6
7 int main ()
8 {
9     cout << "Hello World! ";
10    cout << "I'm a C++ program";
11    return 0;
12 }
```

```
Hello World! I'm a C++ program
```

Note that the result is the same for all the following...

Hello World! I'm a C++ program

```
1 // my second program in C++
2
3 #include <iostream>
4
5 using namespace std;
6
7 int main ()
8 {
9     cout << "Hello World! ";
10    cout << "I'm a C++ program";
11    return 0;
12 }
```

```
int main ()
{
    cout <<
        "Hello World!";
    cout
        << "I'm a C++ program";
    return 0;
}
```

```
int main () { cout << " Hello World! "; cout << " I'm a C++ program "; return 0; }
```

Comments are parts of the source code disregarded by the compiler. They simply do nothing. Their purpose is only to allow the programmer to insert notes or descriptions embedded within the source code.

C++ supports two ways to insert comments:

```
// line comment  
/* block comment */
```

The first of them, known as **line comment**, discards everything from where the pair of slash signs (//) is found up to the end of that same line.

```
1 /* my second program in C++  
2    with more comments */  
3  
4 #include <iostream>  
5 using namespace std;  
6  
7 int main ()  
8 {  
9     cout << "Hello World! "; // prints Hello World!  
10    cout << "I'm a C++ program"; // prints I'm a C++ program  
11    return 0;  
12 }
```

```
Hello World! I'm a C++ program
```



Variables, Data Types and Constants

✓ **Variable** = Portion of memory to which we can store a value and from which we can later retrieve that value.

`a = 5;`

`b = 2;`

`c = a + 1;`

`result = c - b;`

Variable

- ✓ is named storage location in the computer's memory for holding a piece of data



-
- ✓ Information stored in variables is actually stored in RAM
- ✓ Allow you to store and work with data in the computer's memory
- ✓ Named so, because their value(the data they contain) can change while the program is running

Identifier

- ✓ Program defined name that represents some element of a program e.g a variable name
 - ✓ Should give an indication of what the variables are used for or should reflect their purpose
- e.g. number (in the previous figure)

Legal(Valid) Identifiers

- ✓ The first character must be one of the letters **a** through **z**, **A** through **Z**, or an underscore character (**_**).
- ✓ After the first character you may use the letters **a** through **z** or **A** through **Z**, the digits **0** through **9**, or underscores.
- ✓ Uppercase and lowercase characters are distinct. This means **ItemsOrdered** is not the same as **itemsordered**.

Which are legal identifiers?

✓ dayOfWeek

Legal

✓ 3dGraph

Illegal. Variable names cannot begin with a digit.

✓ _employee_num

Legal

✓ June1997

Legal

✓ Mixture#3

Illegal. Variable names may only use letters, digits and

Valid Identifiers

- ✓ White spaces are not allowed in identifiers
- ✓ But can contain mix of upper case and lower case letters
- ✓ The convention is to capitalize the first letter of each subsequent word in identifier leaving all others in small letters

e.g. `itemsOrdered`

- ✓ Variable names are case sensitive

e.g. `itemsOrdered` is not the same as `itemsordered`

Valid Identifiers

- ✓ Another rule is that, when inventing your own identifiers they can not match any keyword of the C++ language nor your compiler specific ones which are reserved keywords

asm	auto	break	bool	case
catch	char	class	const	const_cast
continue	default	delete	do	double
dynamic_cast	else	enum	explicit	extern
false	float	for	friend	goto
if	inline	int	long	mutable
namespace	new	operator	private	protected
public	register	reinterpret_cast	return	short
signed	sizeof	static	static_cast	struct
switch	template	this	throw	true
try	typedef	typeid	typename	union
unsigned	using	virtual	void	volatile
wchar_t	while			

Data types

- ✓ In programming, there are generally two types of data: numbers such as 3, and characters such as the letter 'A'
- ✓ Numbers are used to perform mathematical operations while
- ✓ Characters are used to print information to the screen
- ✓ In programming, the type of data that will be stored in the variable must be known

Data Types

- ✓ Numerical data types can be either integer or whole numbers or real or floating point numbers

E.g. Integers or whole numbers

5, 7, -129 and 32154

And real or floating point numbers

e.g. 3.14159, 6.7, 1.0002

Data Types

- ✓ The computer has to know what kind of data we want to store in them because it doesn't occupy the same amount of memory to store a simple number than to store a single letter or a large number
- ✓ Memory in computers is organized in bytes
 - Byte
 - Minimum amount of memory that we can manage in C++
 - Can store relatively small amount of data i.e. one single character or a small integer (generally an integer between 0 and 255)
- ✓ In addition, the computer can manipulate more complex data types that come from grouping several bytes, such as long numbers or non-integer data types

Data Types

- ✓ **Signed** types can represent both positive and negative values, whereas **unsigned** types can only represent positive values (and zero)

Data Type	Size	Range
short	2 bytes	-32,768 to +32,767
unsigned short	2 bytes	0 to +65,535
int	4 bytes	-2,147,483,648 to +2,147,483,647
unsigned int	4 bytes	0 to 4,294,967,295
long	4 bytes	-2,147,483,648 to +2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295

Considerations for selecting numeric data type

- ✓ The largest and smallest numbers that may be stored in the variable
- ✓ Whether the variable stores signed numbers
- ✓ The number of decimal places of precision the variable has
- ✓ How much memory the variable uses

Guarantees

Size of integer is dependent on the type of system you are using but these are guaranteed to be satisfied:

- ✓ Integers are at least as big as short integers.
- ✓ Long integers are at least as big as integers.
- ✓ Unsigned short integers are the same size as short integers.
- ✓ Unsigned integers are the same size as integers.
- ✓ Unsigned long integers are the same size as long integers.

Char Data Type

- ✓ One byte integer data type
- ✓ Char variable can hold a single character, but strictly speaking it is an integer data type

Strings

- ✓ Allow a series of characters to be stored in consecutive memory locations
- ✓ Can be virtually any length
- ✓ The program knows how long a string is by appending an extra byte to the end of string constants and
- ✓ In this byte the number 0 is stored called null terminator or null character
- ✓ e.g.

S	e	b	a	s	t	i	a	n	\0
---	---	---	---	---	---	---	---	---	----

Variable Declaration

- ✓ Tells the compiler the variable's name and the type of data it will hold
- ✓ The syntax for variable declaration is to write the specifier of the desired data type (like int, bool, float...) followed by a valid variable identifier

e.g. `int a;`

`float myNumber;`

- ✓ Once declared the variables can be used within the rest of the program(must be declared before we use them)

Variable declaration

- ✓ To declare more than one variable of the same type, it can be done in a single statement by separating their identifiers with commas

e.g.

```
1 int a;  
2 int b;  
3 int c;
```

 Can be written as

```
int a, b, c;
```

- ✓ When declaring a variable, its value is initially undetermined
- ✓ Makes a variable to store a concrete value at the same moment that it is declared

Initialization of Variables

```
1 // initialization of variables
2
3 #include <iostream>
4 using namespace std;
5
6 int main ()
7 {
8     int a=5;           // initial value = 5
9     int b(2);          // initial value = 2
10    int result;         // initial value undetermined
11
12    a = a + 3;
13    result = a - b;
14    cout << result;
15
16    return 0;
17 }
```

Assignment(=)

number = 5;

- ✓ The = operator copies the value on its right to the variable names on its left
- ✓ The variable name must always appear on the left
- ✓ Which of the following are legal C++ assignment statements?

a = 7;

legal

7 = a;

Illegal, because the variable name must be on the left

7 = 7;

Illegal, because the left hand signed of the assignment operator is not a variable

Constant

- ✓ Unlike a variable, it is a data item whose value cannot change during the program's execution
- ✓ Includes literals, defined constants and declared constants

Literals

- ✓ Are Used to express particular values within the source code of a program
- ✓ e.g `a = 5;`
- ✓ 5 is a literal constant
- ✓ Literals like variables are considered to have a specific data type
- ✓ Literals can be Integer Numerals, floating point numbers, character and string literals

Literals

Integer Numerals

- ✓ Numerical constants that identify integer decimal values
- ✓ C++ allows use of octal(base 8) and hexadecimal(base 16) numbers as string literals

e.g.

```
1 75          // decimal
2 0113       // octal
3 0x4b       // hexadecimal
```

Literals

Integer Literals

- ✓ are by default of type Int
- ✓ Can be forced to be unsigned by appending the u character to it, or long by appending l

e.g. 75 //int

75u //unsigned int

75l //long

75ul //unsigned long

Literals

Floating point numbers

- ✓ Express numbers with decimals and or exponents

e.g.

```
1 3.14159      // 3.14159
2 6.02e23      // 6.02 x 10^23
3 1.6e-19      // 1.6 x 10^-19
4 3.0          // 3.0
```

- ✓ Default value of floating point literals is *double*
- ✓ To explicitly express a float or long double numeric literal, you can use the *f* or *L* suffixes

E.g.

```
1 3.14159L     // long double
2 6.02e23f     // float
```

Literals

Character and string literals

```
1 'z'  
2 'p'  
3 "Hello world"  
4 "How do you do?"
```

- ✓ The first two are single character constants and the following two are several characters
- ✓ To represent a single character, we enclose it between single quotes(') and to express a string(which generally consists of more than one character), we enclose it in double quotes("")

Literals

String literals

- ✓ can extend to more than a single line of code by putting a backslash sign (\) at the end of each unfinished line.

e.g.

```
1 "string expressed in \  
2 two lines"
```

- ✓ Can concatenate several string constants by separating them by one or several blank spaces

e.g. "this forms" "a single" "string" "of characters"

Escape codes

- ✓ Special characters, that are difficult or impossible to express otherwise in the source code of a program, like newline(`\n`) or

tab(`\t`)

<code>\n</code>	newline
<code>\r</code>	carriage return
<code>\t</code>	tab
<code>\v</code>	vertical tab
<code>\b</code>	backspace
<code>\f</code>	form feed (page feed)
<code>\a</code>	alert (beep)
<code>\'</code>	single quote (')
<code>\"</code>	double quote (")
<code>\?</code>	question mark (?)
<code>\\</code>	backslash (\)

Boolean literals

- ✓ There are two valid boolean values: true and false
- ✓ Can be expressed in C++ as values of type `bool` by using the boolean literals true and false

Defined Constants

✓ can define your

```
1 #define PI 3.14159  
2 #define NEWLINE '\n'
```

own names for
constants that you
use very often
without having to

Declared Constants(const)

- ✓ With const prefix, you can declare constants with a specific type in the same way as you would do with a variable

```
1 const int pathwidth = 100;  
2 const char tabulator = '\t';
```

- ✓ The const qualifier specifies that the value of the variable will not change throughout the program
- ✓ Why bother with using a variable when we can simply place the literal constant in its place everywhere in

Exercise.

List all the variables and constants that appear in the following program.

// This program uses variables and constants

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{ int little;
```

```
    int big;
```

```
    little = 2;
```

```
    big = 2000;
```

```
    cout << "The little number is " << little << endl;
```

```
    cout << "The big number is " << big << endl;
```

```
    return 0;
```

```
}
```

Operators, Expressions and Statements

Statements and Expressions

Statements

- A statement is the smallest independent unit in a C++ program.
- Statements in C++ generally end in semicolons.

Expressions

- Any arrangement of variables, constants and operators that specifies a computation.
- 3.14, 42+3, and val1*(23+ val1) are expressions.

Expressions ...

There are two kinds of expressions in C++:

An expression may appear as either the left-hand or right-hand of an assignment.

Lvalue

✓ An expression *may appear on the left but not on the right-hand* side of an assignment.

Rvalue

✓ An expression *may appear on the right but not on the left-hand* side of an assignment.

Arithmetic Operators

Operator	Name	Usage
+	unary plus	+ expr
-	unary minus	- expr
*	multiplication	expr1*expr2
/	division	expr1/expr2
%	remainder (modulus)	expr1%expr2
+	addition	expr1+expr2
-	subtraction	expr1-expr2

Note: expr stands for any expression to be operated on by the operator. The term operand is also used.

Arithmetic Operators...

- Division between integers results in an integer. If the quotient contains a fractional part, it is truncated.

```
int example = 10/3; //example equals 3, remainder truncated
```

```
int example2 = 9/3; //example2 equals 3, no remainder
```

- The % operator is known as the *remainder or modulus operator*. It *computes the* remainder of dividing the lefthand operand by the righthand operand.

```
int example3 = 10%3; //example3 equals 1, the remainder
```

Arithmetic Operators...

➤ Precedence :

Unary Operators

Multiplication and Division Operators

Remainder Operator

Binary Addition and Subtraction Operators

These operators are all left associative,

For example, $5 + 10 * 20 / 2$, the operations are performed in the following order.

- $10 * 20$ (result = 200)
- $200 / 2$ (result = 100)
- $5 + 100$ (result = 105)

Relational and Logical Operators

- return values of type **bool**.
- used to test certain conditions to find out whether they are true or not.

Operator	Name	Usage
<code>!</code>	Logical Not	<code>!expr</code>
<code><</code>	Less than	<code>expr1 < expr2</code>
<code><=</code>	Less than or equal	<code>expr1 <= expr2</code>
<code>></code>	Greater than	<code>expr1 > expr2</code>
<code>>=</code>	Greater than or equal	<code>expr1 >= expr2</code>
<code>==</code>	Equality	<code>expr1 == expr2</code>
<code>!=</code>	Inequality	<code>expr1 != expr2</code>
<code>&&</code>	Logical AND	<code>expr1 && expr2</code>
<code> </code>	Logical OR	<code>expr1 expr2</code>

Relational and Logical Operators...

- Logical NOT operator (!) *treats its operand as a condition. It yields a result that has the opposite truth value from its operand.*
- Relational operators (<, <=, >, >=, ==, !=) *compare two operands. They return true if the condition for comparison is fulfilled and false otherwise.*

Bitwise Operators

- take operands of integral type.
- treat their integral operands as a collection of bits, providing operations to test and set individual bits
 - ✓ Bitwise AND (&)
 - ✓ Bitwise OR (|)
 - ✓ Bitwise XOR (^)
 - ✓ Bitwise NOT (~)
 - ✓ Left Shift (<<) and Right Shift (>>)

Assignment Operators

- The lefthand operand of an assignment operator (=) must be a ***nonconst lvalue***.

```
int i, j, ival;  
const int ci = i; // ok: initialization not assignment  
1024 = ival;      // error: literals are rvalues  
i + j = ival;     // error: arithmetic expressions are rvalues  
ci = ival;        // error: can't write to ci – const lvalue
```

- Assignments where the types of the left and right operands differ may require conversions that might change the value being assigned

```
int iVar = 0; //result: type int, value 0  
iVar = 3.14; //result: type int, value 3
```

Assignment Operators...

- An assignment returns an lvalue.
- Therefore, we can perform multiple assignments in a single expression.

```
int a, b;  
a = b = 0; //both a and b assigned 0
```

- Unlike the other binary operators, the assignment operators are right associative.

Compound Assignment Operators

```
int sum = 0;  
sum = sum + 10;
```

C++ provides operators that simplify this task

- The compound operators are of the form ***op=*** (*where op is the operator*)

a op= b means a = a op b

Therefore

a += b means a = a + b

a -= b means a = a - b

a *= b means a = a * b

a /= b means a = a / b

a %= b means a = a % b

Increment and Decrement Operators

- Operation of adding or subtracting one from a variable is done by ***increment (++)*** and ***decrement (--)*** operators
- There are two forms of these operators:
 - ***prefix and postfix.***

Prefix increment operation increments its operand and returns the changed value as its result

Postfix version increments the operand but returns the copy of the original unchanged value as its result

Increment and Decrement Operators...

```
int i = 0, j;  
j = ++i;    // j=1, i=1: prefix increment  
j = i++;    // j =1, i = 2: postfix increment  
  
j = --i;    // j=1, i = 1: prefix decrement  
j = i--;    // j=1, i=0: postfix decrement
```

Note: the prefix version returns an lvalue, the postfix version returns an rvalue;

Evaluating Compound Expressions

- ***Compound Expression:*** An expression with two or more operators.
- In a compound expression, the way in which the operands are grouped to the operators may determine the result of the overall expression. If the operands group one way, the result differs from what it would be if they grouped another way
- Precedence and associativity determine which part of the expression is the operand for each of the operators in the expression.
- We can override these rules by parenthesizing compound expressions to force a particular grouping.

Summary of Operators

The list is in order of decreasing precedence.

Operators	Associativity
Postfix increment and decrement (++ , −)	Right
Prefix increment and decrement (++ , −)	Right
!, + (unary plus), - (unary minus)	Right
*, / , %	Left
+, - (arithmetic)	Left
<, <=, >, >=	Left
!=, ==	Left
&&,	Left
+=, -=, *=, /=, %=, =,	Right

Exercise

1. Using the above table, evaluate the following expressions
 - a) $4+7*6/3-5$
 - b) $15/3*34-(5+7)/6$
 - c) $++23/-3+3--$
2. Write a program to verify your answer for question 1.

Statement Classifications

- **Expression Statements:** are statements that compute expressions.

```
42;           // Valid but pointless
cout << 42;    // More typical
x = y * z;     // An assignment
;             // Null statement
```

- **Declaration Statements:** are statements that declare variables.

```
int x, y, z=10;
```

Statement Classifications ...

- **Compound Statements:** A compound statement is a sequence of zero or more statements enclosed within curly braces. Also called a **block**.

```
{  
    int x;  
    cout<<x;  
}
```


3.5. Using Library Functions

Mathematical Functions

The mathematical functions allow us to do mathematical operations.

These operations include:

- raising a number to a certain power,
- computing the square root of a number,
- computing the cosine of an angle, etc... .

These functions are defined in the header file ***math.h*** (or ***cmath*** in standard C++).

Mathematical Functions ...

```
1.  /* Program to calculate the square root of a number */
2.  #include <iostream.h> //for cout and cin
3.  #include <math.h>    //for the sqrt() function
4.
5.  int main()
6.  {
7.      double num, ans;
8.      cout<<"Enter the number: > ";
9.      cin>>num;
10.     ans = sqrt(num); /*calculate the square root of
11.     num and store the result in ans*/
12.     cout<<"The square root of "<<num<<" is "<<ans<<endl;
13.     return 0;
14. }
```

sqrt is the name of the function that performs the square root operation. This function **takes one argument of type double and returns a result of type double.**

Mathematical Functions ...

The function that computes the power of two numbers is :

```
ans = pow(3, 4);
```

More examples of mathematical functions are:

```
ans = cos(num); //calculates the cosine of num (radians)
ans = sin(num); //the sine of num
ans = abs(num); //the absolute value of num
ans = exp(num); //e raised to num
ans = log(num); //the natural logarithm (base e) of num
ans = log10(num); //the common logarithm (base 10) of num
```

IO Manipulators

- IO manipulators are operators used with the insertion operator (<<) to modify or manipulate the way data is displayed.
- We've already seen the ***endl*** *manipulator which* writes a new line character and forces the output to be displayed.
- Other IO manipulators, which take arguments, are defined in the ***iomanip.h*** (or ***iomanip*** in standard C++) header file.

IO Manipulators ...

Here are the most commonly used manipulators:

- ***setw(n)***: Sets the minimum size of a field to *n*. Applies only to the next element applied to the output.
- ***left***: Left justifies the output. Only useful after ***setw(n)***.
- ***right***: Right justifies the output. Only useful after ***setw(n)***. (Default)
- ***setfill(ch)***: Only useful after ***setw(n)***. If a value does not entirely fill a field, the character *ch* will be used to fill in the other characters.
- ***setprecision(n)***: Sets the number of digits printed to the right of the decimal point. This applies to all subsequent floating point numbers written to that output stream.
- ***fixed***: Uses fixed point notation for floatingpoint numbers
- ***scientific***: Formats floatingpoint numbers in scientific notation.

IO Manipulators ...

Observe the output of the following program (*this program uses standard C++*)

```
1. #include <iostream>
2. #include <iomanip>
3. using namespace std;
4. int main()
5. {
6.     cout<<"Columns Demonstration"<<endl;
7.     //use setw(10) to create columns (fields) 10 characters wide
8.     cout<<setw(10)<<"Column 1"<<setw(10)
9.         <<"Column 2"<<setw(10)<<"Column 3"<<endl;
10.    //second row, simple set of data
11.    cout<<setw(10)<<1<<setw(10)<<2<<setw(10)<<3<<endl;
12.    //third row, left aligned
13.    cout<<left<<setw(10)<<4<<setw(10)<<5<<setw(10)<<6<<endl;
14.    //fourth row, remaining space filled with *
15.    cout<<setfill('*')<<setw(10)<<7
16.        <<setw(10)<<8<<setw(10)<<9<<endl;
17.    //fifth row, floating point numbers:
18.    //precision 3, right aligned and remaining spaces unfilled
19.    cout<<setprecision(3)<<right<<setfill(' ')<<setw(10)
20.        <<9.010023<<setw(10)<<9.020023<<setw(10)<<9.030023<<endl;
21.    return 0;
22. }
```

Exercises

1. Write a program that prints a table showing the sin, cos, and tan values for the uncommon angles (15, 20, 70, and 140). Make sure that the values are displayed only up to 4 significant digits.
2. The general characteristics of a semiconductor diode can be defined by the following equation

$$I_D = I_S (e^{kV_D/T_K} - 1)$$

where $k = 11,600$ K/V for Silicon, V_D is the diode voltage in Volts, T_K is the temperature in Kelvins, I_S is the saturation current in Amperes, and I_D is the diode current.

Write a program that accepts I_S (ranging from 10^{-15} A upto 10^{-9} A), V_D (ranging from 0 to 1Volts) and T_K (ranging from -50 °C to 150 °C) from the user and calculates I_D .

3. Modify the above program to print a table for the voltages and currents for a given temperature (obtained from the user). Take $I_S = 10^{-9}$ A and calculate I_D for $V_D = 0, 0.1, 0.2, \dots, 1$ V. Do these values agree with the diode curve you saw in electronics class?

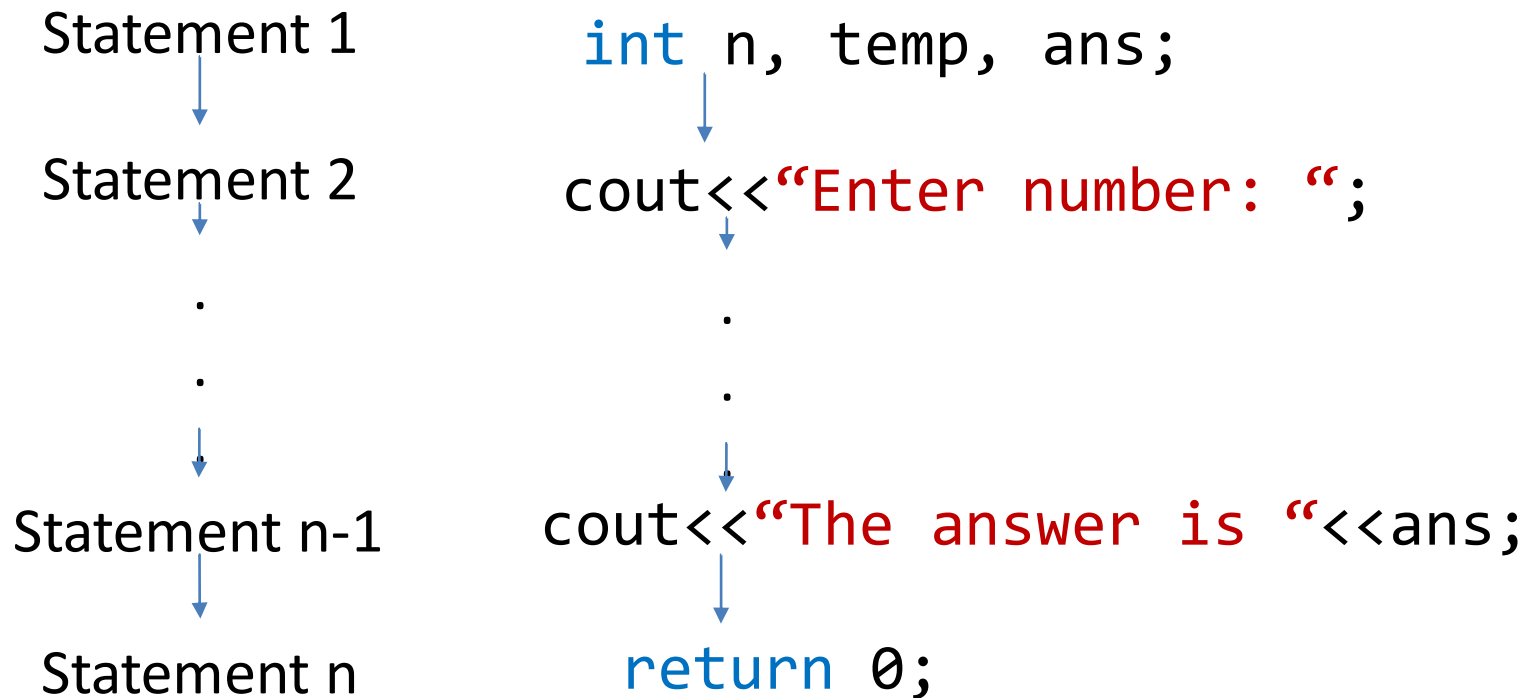


Chapter 3

Controls & Decisions

So Far...

The C++ programs we have been writing so far have been executing sequentially



Flow of Control

- A program is usually not limited to a linear sequence of instructions. During its process it may **repeat code (Looping)** or take decisions/select (selection). For that purpose, C++ provides control structures that serve to specify what and how have to perform our program.

1. Decisions/ Selection (if, if...else, switch)

- Using decision or selection statement, we select one block of code based on certain conditions.

i) If statements

- ***An if statement has two forms:*** one with an else branch and one without

i) *if* statement (single selection)

Syntax:

If(expression or condition)

```
{  
//code/statement(s)  
}
```

A test expression that evaluates to a boolean value (either true or false)

A single statement or a compound statement, to be executed if ***condition*** evaluates to *true*.

If the ***condition*** is *true*, the ***statement*** is executed, otherwise it is skipped.

Example 1

```
unsigned short age;
```

```
Enter your age:> 12
You are too young
Thank you
```

```
cout<<"Enter your age:";
```

```
cin>>age;
```

```
if (age < 17)
```

```
Enter your age:> 56
Thank you
```

```
cout<<"You are too young!\n";
```

```
cout<<"Thank you."<<endl;
```

Note: the if statement in this example, executed only a *single*

Example 2

- In order to execute multiple statements, we can use a block e.g.

```
int val;  
cout<<"Enter an integer value: "<<endl;  
cin>>val;  
if (val%2 == 0)  
{  
    int quotient = val/2;  
    cout<<val<<" is divisible by 2."<<endl;  
    cout<<"The quotient is "  
}  
cout<<"Thank you."<<endl;
```

```
Enter an integer value:> 8  
8 is divisible by 2  
The quotient is 4  
Press any key to continue...
```

```
Enter an integer value:> 7  
Press any key to continue...
```


ii) *if... else* statement (double selection)

Syntax:

If(expression or condition)

```
{  
  //code1/statement1  
}
```

If ***condition*** evaluates to *true*, this statement will be executed

else

```
{  
  //code2/statement2  
}
```

If ***condition*** evaluates to *false*, this statement will be executed

Example 1

Note: the if and else statements executed a single statement if the condition is true or false

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int x;

    cout<<"Enter a number: ";
    cin>> x;

    if(x>10)
        cout<<x<<" is greater than 10"<<endl;
    else
        cout<<x<<" is not greater than 10"<<endl;

    system("pause");
    return 0;
}
```

```
Enter a number: 7
7 is not greater than 10
Press any key to continue . . .
```

```
Enter a number: 12
12 is greater than 10
Press any key to continue .
```

Example 2

In order to execute multiple statements we can use a **block**

```
#include <iostream>
using namespace std;

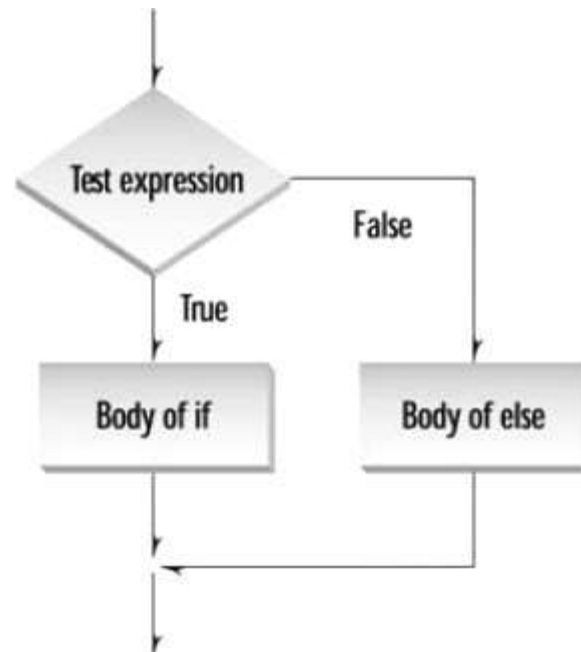
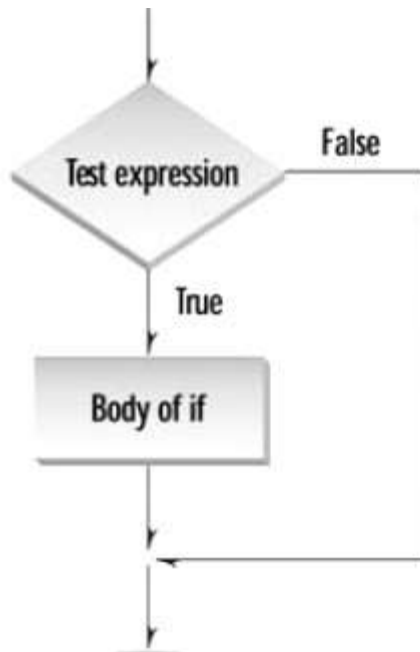
int main()
{
    int x;

    cout<<"Enter a number: ";
    cin>> x;

    if(x>10)
    {
        //both statements will be executed if x>10
        cout<<"You entered "<<x<<endl;
        cout<<x<<" is greater than 10"<<endl;
    }
    else
    {
        //both statements will be executed if x<10
        cout<<"You entered "<<x<<endl;
        cout<<x<<" is not greater than 10"<<endl;
    }

    system("pause");
    return 0;
}
```

Flowchart equivalents



The Conditional Operator

- Is another short-hand method of expressing if...else statements.

Syntax:

condition ? expression1 : expression2;

An expression that evaluates to a boolean.

Expression to execute if ***condition*** evaluates to *true*.

Expression to execute if ***condition*** evaluates to *false*.

Example

```
(x < y) ? x = 10 : y = 10;
```

//this is the same as

```
if (x < y)
```

```
    x = 10;
```

```
else
```

```
    y = 5;
```

iii) Nested if ...else statements

```
If(expression)
    statement1
else
    statement2
```

Note that: statement1 and statement 2 can be any statement including **if... else** statement

Nested if ...else statements

E.g. when statement 2 itself is again another if...else statement

```
#include <iostream>
using namespace std;
```

```
int main()
{
```

```
    int x;
```

```
    cout<<"Enter a number: ";
```

```
    cin>> x;
```

```
    if(x>10)
```

```
        cout<<x<<" is greater than 10"<<endl;
```

```
    else
```

```
        if(x<5)
```

```
            cout<<x<<" is less than 5"<<endl;
```

```
        else
```

```
            cout<<x<<" is between 5 and 10"<<endl;
```

```
    }
```

```
    system("pause");
```

```
    return 0;
```

```
}
```

```
Enter a number: 12
12 is greater than 10
```

```
Enter a number: 3
3 is less than 5
```

```
Enter a number: 7
7 is between 5 and 10
```


If ...else if ... else

Convenient form for nested *if...else* statements

```
if (condition1)  
    statement1  
else if (condition2)  
    statement2  
...  
  
else if (conditionN-1)  
    statementN-1  
else  
    statementN
```

Example :

```
unsigned short age;  
cout<<"Enter your age: ";  
cin>>age;  
if (age < 17)  
    cout<<"You are too young!\n";  
else if (age < 40)  
    cout<<"You are still young!\n";  
else if (age < 70)  
    cout<<"Young at heart!\n";  
else  
    cout<<"Are you sure?\n";  
  
cout<<"Thank you."<<endl;
```

//This program illustrates a bug that occurs when independent if/else statements are used to assign a letter grade to a numeric test score.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int testScore; char
```

```
grade;
```

Program Output with Example Input Shown in Bold

Enter your numeric test score and I will tell you the letter grade you earned: **40**[Enter]
Your grade is A.

//It can be corrected like
this

```
#include <iostream>
using namespace
std;
```

Program Output With Example Input Shown in
Bold

Enter your numeric test score and I will tell you
the letter grade you earned: **40**[Enter]

num grades

Dangling else/ matching else problem

```
#include <iostream>
using namespace std;

int main()
{
    int x;

    cout<<"Enter a number: ";
    cin>> x;

    if(x>10)
        if(x<20)
            cout<<x<<" is between 10 and 20"<<endl;
        else
            cout<<x<<" is greater than 20"<<endl;
    |
    system("pause");
    return 0;
}
```

The above program introduces a source of potential ambiguity called a **dangling else** problem

Dangling else problem

- Is the else statement in the above program matched up with the outer or inner if statement?
- The answer is that an else statement is paired up with the ***last unmatched*** if statement in the **same** block

Dangling else problem

(The above program can be written without ambiguity like this)

```
#include <iostream>
using namespace std;

int main()
{
    int x;

    cout<<"Enter a number: ";
    cin>> x;

    if(x>10)
    {
        if(x<20)
            cout<<x<<" is between 10 and 20"<<endl;
        else //attached to inner if statement
            cout<<x<<" is greater than 20"<<endl;
    }

    system("pause");
    return 0;
}
```

Dangling else problem

- However, if we want the else to attach to the outer if statement, we can do this explicitly by enclosing the inner if statement in a block

```
#include <iostream>
using namespace std;

int main()
{
    int x;

    cout<<"Enter a number: ";
    cin>> x;

    if(x>10)
    {
        if(x<20)
            cout<<x<<" is between 10 and 20"<<endl;
    }
    else //attached to inner if statement
        cout<<x<<" is less than 10"<<endl;

    system("pause");
    return 0;
}
```


Summary on Dangling else problem

- The use of a block tells the compiler that the else statement should attach to the if statement before the block
- Without the block, the else statement would attach to the nearest unmatched if statement, which would be the inner if statement

6. Write an if-else statement that outputs the word High if the value of the variable score is greater than 100 and Low if the value of score is at most 100. The variable score is of type int.

ii) The Switch Statement

- It is usually very difficult to express our intended logic using such deeply nested *if...else* statements.
- As an alternative method of choosing among a set of mutually exclusive choices, C++ provides the *switch* statement.

iv) The *switch* Statement

Syntax:

```
switch (expression)
{
```

```
    case constant1:
        statements
        break;
```

```
    case constant2:
        statements
        break;
```

...

```
    case constantN:
        statements
        break;
```

An integer expression, whose value we want to test

Unique, possible values that we are testing for equality with *expression*

Statements to be executed when *expression* becomes equal to *constantN*

Statements to be executed when *expression* is not equal to any of the N constants

The break statement causes the Program to exit the switch statement

The Switch Statement

- The switch expression is evaluated to produce a value,
- and each **case** label is tested against this value for equality.
- If a case label matches, the statements after the case label are executed.
- If no case **label** matches the switch expression, the code under the default label is executed (if it exists).

Example

```
// This program demonstrates the use of a switch statement.  
// The program simply tells the user what character they entered.
```

```
#include <iostream>  
using namespace std;  
int main()  
{  
    char choice;  
    cout << "Enter A, B, or C: ";  
    cin >> choice;  
    switch (choice)  
    {  
        case 'A': cout << "You entered A.\n";  
        break;  
        case 'B': cout << "You entered B.\n";  
        break;  
        case 'C': cout << "You entered C.\n";  
        break;  
        default: cout << "You did not enter A, B, or C!\n";  
    }  
    return 0; }
```

```
Enter A, B, or C: B  
You entered B.  
Press any key to continue . . .
```

```
Enter A, B, or C: F  
You did not enter A, B, or C!  
Press any key to continue . . .
```

The switch statement

```
int x;
cout<<"Enter a number";
cin>>x;
switch (x)
{
    case 1:cout<<"You Entered 1:";
        break;
    case 2:cout<<"You Entered 2:";
        break;
    case 3:cout<<"You Entered 3:";
        break;
    default: cout<<"Invalid Input:";
        break; }
```

- Note that the expressions of each case statement in the block must be unique.
- That is , you can't do this

```
Switch(x)
{
    case 4:
        ....
    case 4://illegal...already used
        value 4
        .....
    default:
```

Break Statement

Without the break statements, program execute all of the lines from the

```
Enter A, B, or C: A
You entered A.
You entered B.
You entered C.
You did not enter A, B, or C?
Press any key to continue . . .
```

```
Enter A, B, or C: C
You entered C.
You did not enter A, B, or C?
Press any key to continue . . .
```


Break Statement

- When a case is matched (or the default is executed), execution begins at the first statement following that label and continues until one of the following conditions is true:
 - 1) The end of the switch block is reached
 - 2) A break statement occurs

Note that if none of these conditions are met, cases will overflow into other cases! The program “falls through” all of the statements below the one with matching case expression

E.g.

```
switch (2)
{
    case 1: // Does not match -- skipped
        cout << 1 << endl;
    case 2: // Match! Execution begins at the next statement
        cout << 2 << endl; // Execution begins here
    case 3:
        cout << 3 << endl; // This is also executed
    case 4:
        cout << 4 << endl; // This is also executed
        // Break;
    default:
        cout << 5 << endl; // This is also executed
}
```

2
3
4
5

Break Statement

- Sometimes, you may want the program to “fall-through” statements.

2. Looping

- A *loop* is a control structure that causes a statement or group of statements to repeat.
- C++ has three looping control structures: the **while** loop, the **do-while** loop, and the **for** loop.
- The difference between each of these is how they control the repetition.

i)While Loop

- Syntax :

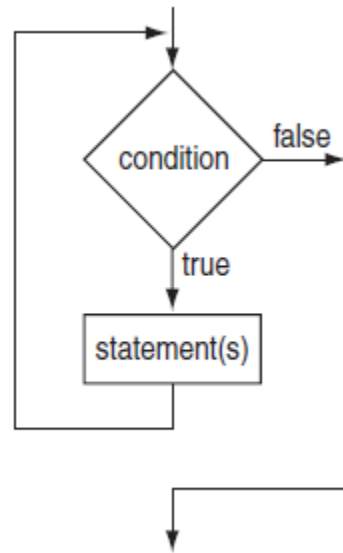
```
while (expression/condition )  
{  
    statement;  
    statement;  
    // Place as many statements here as necessary  
}
```

and its function is simply to repeat *statement* while *expression* is true, Each repetition is known as an *iteration*

Principle:

Condition of while loop is first executed (**pretest loop**) and if condition is TRUE, *statement* is

While Loop Statement



- (If there is only one statement in the loop body, the braces may be omitted.)

```
while (expression)  
    statement;
```

Example

```
int i = 0;
while(i<10)
{
    cout<<i<<" ";
    i++;
}
cout<<"done!";
```

This outputs:

0 1 2 3 4 5 6 7 8 9 done!

While Loop Statement

- An important characteristic of the while loop is that the loop will **never iterate if the test expression is false to start with**.
- It is possible that a while statement executes 0 times.

```
int i = 15;  
while(i<10)  
{
```

```
    cout<<i<<" ";
```

Example-2

// Loop through every
number between 1
and 50

```
int i= 1;
```

```
while (i<= 50)
```


...cont'

// a program that sums numbers
from one up to 10

```
#include<iostream>
using namespace std;
int main()
{
    int n=10, sum=0;
    while(n!=0)
    {
        sum=sum+n;
        n--;
    }
    cout<<"sum="<<sum<<endl;
    return 0;}
```

//a program to print even numbers
between 0 and n

```
#include<iostream>
using namespace std;
int main()
{ int n;
  int i=0;
  Cout<<"enter an integer n:"<<endl;
  Cin>>n;
  While(i<=n)
  {
    if((i%2)==0)
    {
        Cout<<i<<"is even"<<endl;
    }
    i++;
  }
  return 0;
}
```

Infinite Loop

- In all but rare cases, loops must contain within themselves a way to terminate.
- This means that something inside the **loop must eventually make the test expression false**.
- If a loop does not have a way of stopping, it is called an *infinite loop*.
- If the test expression always evaluates to true, the while loop will execute forever until the program is interrupted.

This version of the loop will stop after it has executed 10 times:

```
int i = 0;
while(i<10)
{
    cout<<i<<" ";
    i++;
}
cout<<"done!"
```

Infinite Loop

- Another common pitfall with loops is accidentally using the = operator when you intend to use the == operator.

while (remainder = 1) // Error: Notice the assignment.

{

 cout << "Enter a number: ";

 cin >> num;

 remainder = num % 2;

ii) Do While Loop

- The do-while loop looks similar to a while loop turned upside down

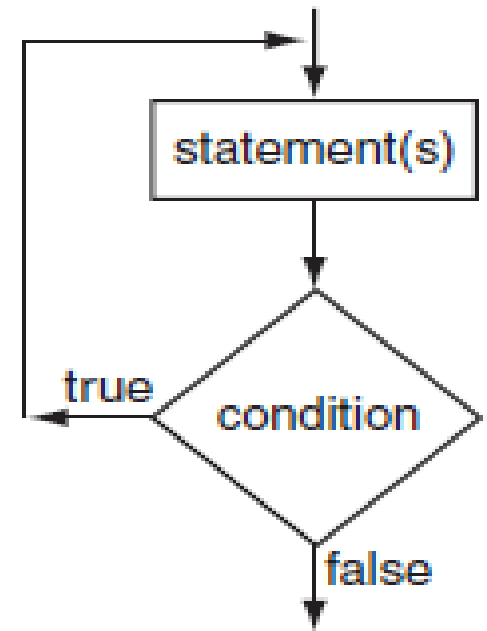
Do

{

statement;

statement;

} while (*expression*);



- Principle: first *Action* is executed. Then if the condition is while is TRUE ,action is repeated and it continues until the condition gets FALSE

//program that displays list of integer from 0 to number n, as:

0,1,2,3,4,5,.....,n.(n is inputted from keyboard)

```
#include<iostream>
using namespace std;
int main()
{
    int n ,i=0;
    cout<<"enter  n:"<<endl;
    cin>>n;
    do
    {
        cout<<i<<","<<endl;
        i++;
    }
    while(i<=n);
    return 0;
}
```

Tuesday, August 26, 2025

//a program that checks if a number input was even or odd and after checking, it asks a user whether to try another number to be checked or not

```
#include<iostream>
Using namespace std;
int main()
{
    int n;
    char mychar;
    do
    {
        Cout<<"enter n:"<<endl;
        Cin>>n;
        if((n%2)==0)
        {
            Cout<<n<<"is even"<<endl;
        }
        else
        {
            Cout<<n<<"is odd"<<endl;
        }
        Cout<<"do you want to try again(y/n)?"<<endl;
        Cin>>mychar;
    }
    While((mychar=='y' || (mychar=='Y')));
    return 0;
}
```

Do While Loop

- The difference between the do-while loop and the while loop is that do-while is a *post-test* loop.
- This means do-while always performs at least one iteration, even if the test expression is false from the start.

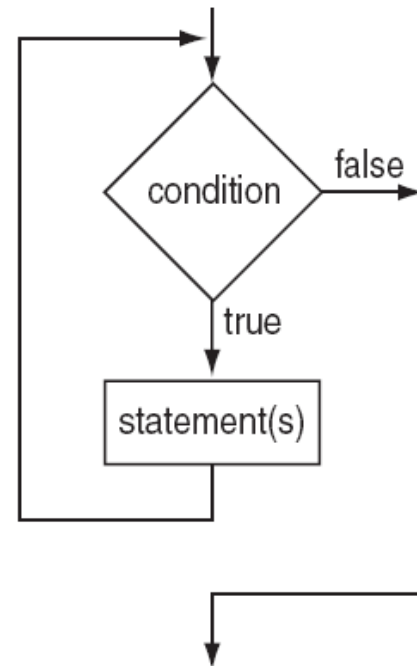
• Ex.

While	Do While
<pre>int x = 1; while (x < 0) cout << x << endl;</pre>	<pre>int x = 1; do cout << x << endl; while (x < 0);</pre>
Never executes	Executes once because the expression <code>x < 0</code> is not evaluated until the end of the loop

iii)For loop

Syntax

```
for (init-statement; test; update)  
{  
    statement;  
    statement;  
    // Place as many statements  
    // here as necessary.  
}
```



For Statements

- The for loop has three expressions inside the parentheses, separated by semicolons.
- The first expression is the *initialization expression*.
It is typically used to initialize a counter or other variable that must have a starting value. This is the first action performed by the loop and it is only done once.
- The second expression is the *test expression*.
Like the test expression in the while and do-while loops, *it controls the execution of the loop.*

For Statements

- As long as this expression is true, the body of the for loop will repeat.
- The for loop is a **pretest loop**, so it evaluates the test expression before each iteration.
- The third expression is the ***update expression***.
It executes at the end of each iteration.
Typically, it increments a counter or other variable that must be modified in each iteration.

Example

```
for(int i = 0; i < 10; i++)
```

```
    cout << i << " ";
```

Consequently, this program prints the result:

0 1 2 3 4 5 6 7 8 9

- Programmers love for loops because they are a very compact way to do loops of this nature
- The previous program can be un-compacted into its while-statement equivalent

```
int i = 0;
```

```
while (i < 10)
```

- Example-1

```
int main()
{
    for(int x=0;x<10;x++)
    {
        cout<<x;
        cout<<endl;
    }
    return 0;
}
```

Out put

0
1
2
3
4
5
6
7
8
9

- Example-2

```
int main()
{
    int sum=0;
    for (int x=1;x<=5;x++)
    {
        sum+=x;
    }
    cout<<sum;
    return 0;
}
```

```
//For looping
#include<iostream>
using namespace std;
int main()
{
    int i;
    for(i=0;i<100;i+=2)
        cout<<i<<endl;
    return 0;
}
```

```
#include<iostream>
using namespace std;
int main()
{
    int i;
    for(i=100;i>0;i--)
        cout<<i<<endl;
    return 0;
}
```

Multiple Declarations

- Although for loops typically iterate over only one variable, sometimes for loops need to work with multiple variables.
- When this happens, the programmer can make use of the comma operator in order to initialize or change the value of multiple variables: E.g.

```
for (int i=0, j=9; i < 10; i++, j--)  
    cout << i << " " << j << endl;
```

Multiple Declarations

- The previous program produces the result

0	9
1	8
2	7
3	6
4	5
5	4
6	3
7	2
8	1
9	0

Nested Loops

- It is also possible to nest loops inside of other loops

// Loop between 1 and 5

- In the following example, the inner loop and outer loops each have their own counters.

```
int i=1;  
while (i<=5)
```

```
{
```

```
    // loop between 1 and i
```

```
    int j = 1;
```

```
    while (j <= i)
```

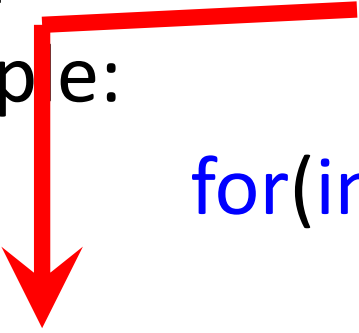
```
        cout << i++;
```

3) Jump statements

i) *break* statement

when executed within *for*, *while*, *do while*, *switch*, it causes an immediate exit from the loop.

Example:



```
for(int i=1;i<=10;i++)  
{  
    if(i==5)  
        break ;  
    cout<<i<<endl;  
}
```

Out put

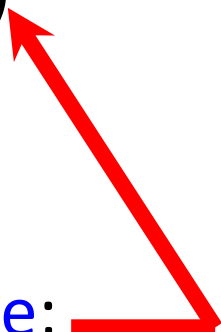
1
2
3
4

continue statement

- when executed within *for*, *while* and *do...while*, the next iteration is done ignoring the rest of the statements

Example:

```
for(int i=1;i<=10;i++)  
{  
    if(i==5)  
        continue;  
    cout<<i<<endl;  
}
```



Out put

1
2
3
4
6
7
8
9
10

iii) goto statement :

Allows to make an absolute jump to another point in the program, the destination point is identified by label(valid identifier followed by a colon(:)).

Example:



```
#include<iostream.h>

int main ( )
{
    int n =10;
    loop:
    goto loop;
```

How to solve problem

- ❖ Develop the algorithm.
- ❖ Convert to cpp code.
- ❖ Check the result.

Example:

- 1) Prime number;
- 2) Grading system;
- 3) Simple calculator; etc

Thank You!

CHAPTER 4

Arrays and Strings

Objectives

- ❖ To use the array data structure to represent lists and tables of values.
- ❖ To define an array, initialize an array and refer to individual elements of an array.
- ❖ To define symbolic constants.
- ❖ To use arrays to store, sort and search lists and tables of values.
- ❖ To define and manipulate multidimensional arrays.
- ❖ To pass arrays to functions.
- ❖ C++ strings

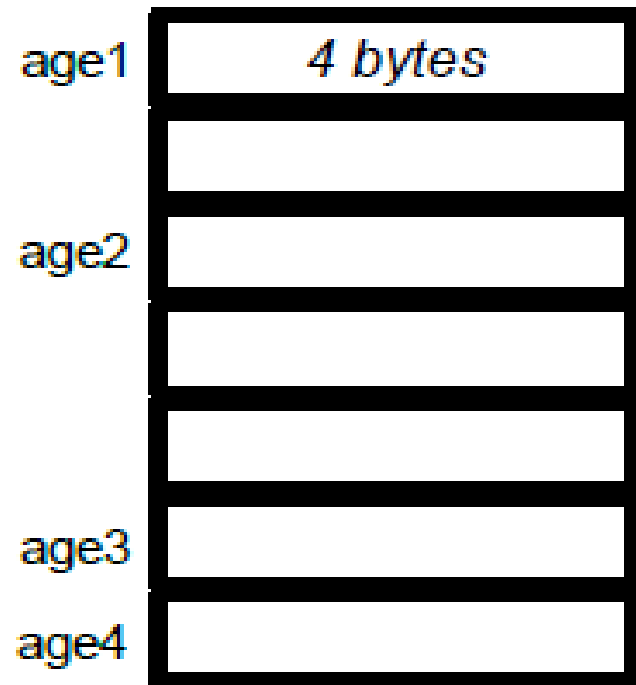
Arrays

Literal way

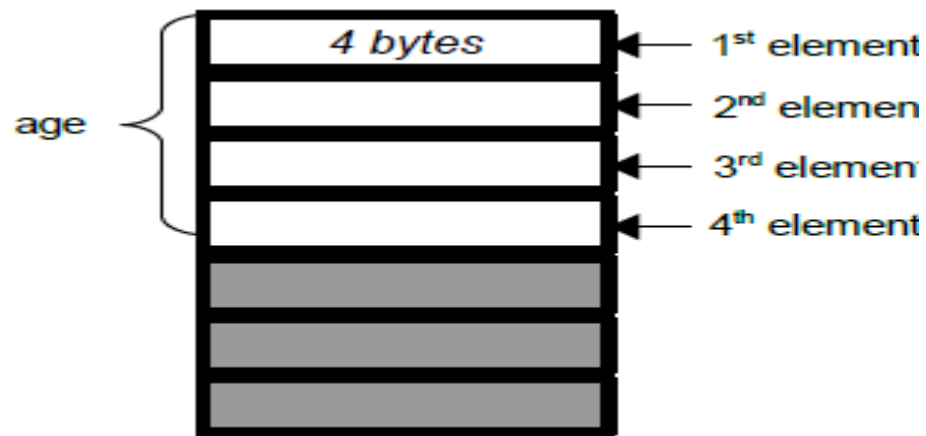
Example:

```
int age1;  
int age2;  
int age3;  
int age4;  
cout<<"Enter ages of five students: ";  
cin>>age1>>age2>>age3>>age4;  
...
```

Memory case



ARRAYS: Are series of elements of the same type placed in contiguous/ continuous memory locations that can be individually referenced by adding an index to a unique name.



Declaration of arrays in C++:

Type *array_name* [*array_size*];

ARRAY_SIZE: specifies how many elements the array has to contain.

int data[5]; // 5 elements all are integers

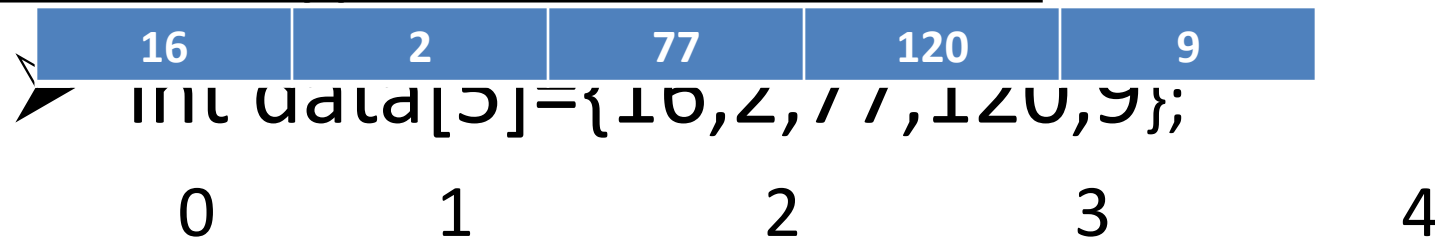
char ch[10]; // 10 elements all are characters

double x[100]; // 100 elements all are double

Initializing array in c++.

❑ If an array is declared as global variable it will be automatically initialized to zero , other wise all the elements will be garbage values.

❑ Various types of initializations:



data

➤ `int data[ ]={16,2,77,120,9};` // we can leave the⁶

❖ Initialization is not the same as an assignment.

❖ Arrays can be initialized, but they **cannot be assigned**:

➤ `float a[7] = { 22.2,44.4,66.6 };`

➤ `float b[7] = { 33.3,55.5,77.7 };`

➤ `b=a;` //ERROR: arrays cannot be assigned!

❖ **Nor can an array be used to initialize another array:**

➤ `float a[7] = { 22.2,44.4,66.6 };`

➤ `float b[7] = a;` // ERROR: arrays cannot be used as initializers!

Accessing the values of an array.

Elements of array are accessed using index.

Syntax: `name[index]`

NB: In c++ indexing always starts from 0.

For the above array the name which we can use to refer to each element is the following:

`da[5] = {16,2,77,120,9};`

<code>da[0]</code>	<code>da[1]</code>	<code>da[2]</code>	<code>da[3]</code>	<code>da[4]</code>
16	2	77	120	9

//to store the value 75 in the third element of
data

data[2] = 75; and,

// to assign/pass the value of the third element of
data to a variable called a, we could write:

a=data[2];

//A program calculate the sum and average of maximum of 20 numbers entered by the user.

```
#include<iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
const int max=20;
```

```
int data[max];
```

```
    int count=0,n;
```

```
    float sum=0,average;
```

```
    cout<<"how many element do want to enter\? \n";
```

```
    cin>>n;
```

```
    if(n<=0||n>20)
```

```
        cout<<"you entered invalied number\n";
```

```
    else
```

```
    {
```

```
        cout<<"enter your data\n";
```

```
        for(int i=0;i<n;i++)
```

```
        {
```

```
            cout<<"data["<<i<<"]=";
```

```
                                cin>>data[i];
```

```
                                sum+=data[i];
```

```
        }
```

```
        average=sum/n;
```


Operations on Arrays

- ❖ Arithmetic operations(+,-,*,/)
- ❖ looking for a value held by one of array members.
- ❖ To find minimum and maximum value of a list
- ❖ To sort some list

Searching Array

```
#include <iostream.h>
```

```
int main()
```

```
{
```

```
    int numbers[] = {10, 25, 36, 44, 52, 60, 75,  
    89}; // Declare the
```

```
    //members of the array
```

```
        int find, i, m = 8;
```

```
    cout << "Enter a number to search:";
```

```
    cin >> find;
```

Multidimensional Array

=> are arrays of arrays or arrays of two or more dimension

=> Used to work with data requiring ***multidimensional arrays*** - for example, matrices and data in the form of a table.

data in the form of a table.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 3 & 5 & 7 \end{bmatrix}$$

Grade Sheet

Name	ECEG-1202	ECEG-1204	ECEG-2207
Abebe Kebede	C	B	A-
Chala Chebude	A	A	B+
Felekech Belete	D	B	C-

Declaration `int a[3][5];` // 2D array

`Char x[3][5][9]` //3D array

Example

- To define a 2D array, *two size specifiers* are required: the *first one* is for the number of rows and the *second one* is for the number of columns.

Eg. `float scores[3][4];`

- To access a certain element **indices or subscript** corresponding to each dimension should be supplied.

- Example

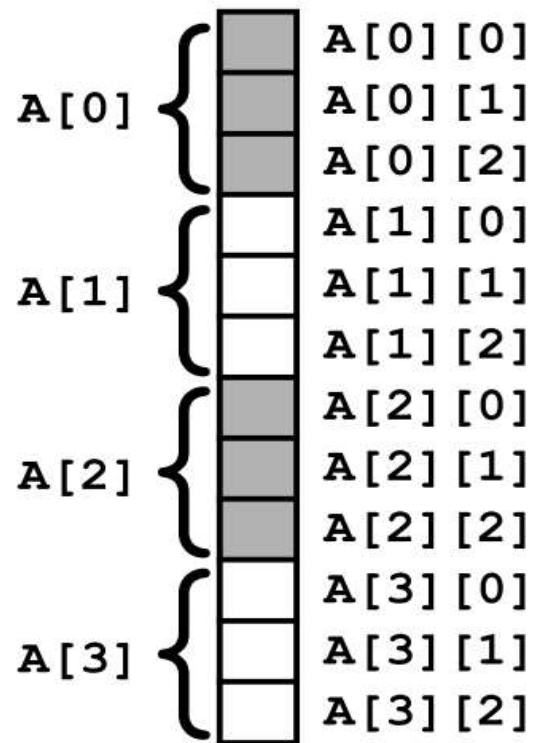
	Column 0	Column 1	Column 2	Column 3
Row 0	<code>score[0][0]</code>	<code>score[0][1]</code>	<code>score[0][2]</code>	<code>score[0][3]</code>
Row 1	<code>score[1][0]</code>	<code>score[1][1]</code>	<code>score[1][2]</code>	<code>score[1][3]</code>
Row 2	<code>score[2][0]</code>	<code>score[2][1]</code>	<code>score[2][2]</code>	<code>score[2][3]</code>

2-D Memory Organization

```
char A[4][3];
```

A is an array of size 4.

Each element of A is
an array of 3 chars



❖ Each element in a 2D array is accessed with two subscripts: the first one is for its row and the second for its column.

Various Initialization options:

```
int a[2][3]={ {1,5,7},{2,3,6}};
```

```
int a[2][3]={0};
```

```
int a[2][3]={ {1,5},{2}}
```

2-D Array Example

```
#include<iostream>
using namespace std;
int main()
{
    const int Numstudents=10;
    const int NumHW=3;
    double grades[Numstudents][NumHW];
    for(int i=0;i<Numstudents;i++)
    {
        for(int j=0;j<NumHW;j++)
        {
            cout<<"Enter HW"<<j<<"Grade for Student Number"<<i<<endl;
            cin>>grades[i][j];
        }
    }
    return 0;
}
```


**Example: write a
program that add two
matrices, input by the
user**

```
#include<iostream>
```

```
using namespace std;
```

```
const int max_row=50;
```

Passing Array as an Argument to Function

Passing elements of Array as an argument(one dimensional)

- An indexed variable can be an argument to a function in exactly the same way that any variable of the array base type can be an argument

double i, n, a[10];

❖ Passing The Entire Array as an argument
(one dimensional)

❖ Passing The Entire Array as an argument
(Two dimensional)

C++ Strings

What Are Strings?

- Strings in C++ are a sequence of characters.
- This new class of objects is nothing you haven't seen before. The following are both examples of strings:

```
"Hello World\n"
```

```
"xyz 123 *&^# $!"
```

- Now we are able to manipulate these literals... but first we must

```
#include <string>
```

Strings as Variables

- To create a variable of type string, simply:

```
string fullName;
```

- Assignment, as always is the same:

```
fullName = "Abebe Kebede";
```

- Or like before, we could always combine the two with an initialization:

```
string fullName = "Abebe Kebede";
```

Input/Output with Strings

- I/O with Strings are as before:

```
string fullName = "";
```

```
cout << "Please enter your name: ";
```

```
cin  >> fullName; //get the name
```

```
cout << "Your name is " << fullName;
```

String Operators: Assignment

- Assignment (=): As with before, assign a string to a variable of type string.

```
string oneName = "Abebe";  
string anotherName = oneName;
```

- Both now hold "Abebe"


```
char name[20];
```

N.B 1.we need not initialize all the available space
(20)

2.the string will terminate by the null character
'\0'

❑ Initialization

```
char my_string[]={ 'H', 'e', 'l', 'l', 'o', '\0' };
```

```
or char my_string[]="Hello"
```

once initialized we can not use the following:

```
my_string[]="hello";
```

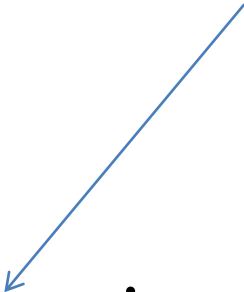
```
my_string="Hello";
```

```
my_string[]={ 'H', 'e', 'l', 'l', 'o', '\0' }.
```

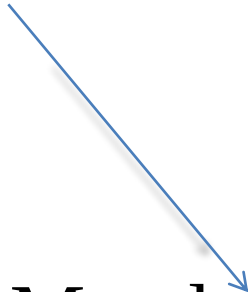
With **cin space** will not be read.

Hence use the following

```
cin.getline(char buffer[], int )
```



Address to store input



Max length

Eg. // cin with strings

```
#include <iostream>
#include <string>
using namespace std;
int main ()
{
    char  mystr[100];
    cout<<"What's your name? ";
    cin.getline(mystr,100);
    cout<<"Hello "<<mystr<<"\n";
    cout<<"What is your favorite team? ";
    cin.getline(mystr,100);
    cout<<"I like "<<mystr<<" too!\n";
    return 0;
}
```

String Manipulations:

Functions to manipulate strings:

Requires string header file in standard c++ and string.h in pre-standard c++.

i)strlen (char buffer[]) =>returns the length without the null character ‘\0’;

```
#include <iostream>
```

```
#include <string.h>
```

```
using namespace std;
```

```
int main ()
```

String concatenation

ii) `strcat(str1, str2)` => to concatenate or merge two strings

NB. 1. The first string `str1` hold the concatenated string

2. the size of the first string must be large enough to hold both strings.

```
#include <iostream>
```

```
#include <string.h>
```

```
using namespace std;
```

```
int main ()
```

```
{
```

String compare

iii) strcmp(str1, str2) => for lexical/alphabetical comparison rule:

- **0** if the two strings are equal
- **negative** if the first string comes before the second in alphabetical order
- **positive** if the first string comes after the second in alphabetical order

String copy

iv) `strcpy(str1, str2)`: is used to copy one string to another. This is because arrays can't be copied using the assignment operator (=).

Example:

```
char str1[20];  
char str2[] = "Second String";  
strcpy(str1, str2);  
cout<<"str1: "<<str1<<endl;  
strcpy(str2, "Another String");
```

String/Numeric Conversion

i)atoi→ string to int	i)itoa → int to string
ii)atof→string to float	ii)ftoi float to string
iii)atoll→string to long	iii)ltoa long to string

Eg.

❑ `int num = atoi("4123"); //num = 4123`

❑ `long lnum = atol ("12345678");`

❑ `float fnum = atof ("2.34");`

❑ `itoa` (not standard): converts an integer to a string.

It accepts three arguments

argument1 - the integer to be converted

argument2 - the string to hold the converted
result

argument3 - the base (8, 10, or 16)

`char strnum[4];`

`itoa(102, strnum, 10);`

`length()` **and** `size()`

- This method returns the integer length of the string. The `length()` **and** `size()` are the same.

- Example:

```
string s1 = "Super!";
```

```
//the integer 6 willbe output.
```

```
cout << s1.length() << endl;
```

at (index)

- This method returns the character at the specified index. Indices start from 0.

- **Example:**

```
string n = "Kebede";
```

```
//the character 'K' will be output.
```

```
cout << n.at(0) << endl;
```

```
//the character 'e' will be output.
```

```
cout << n.at(n.size()-1) << endl;
```

Example Using at

```
#include<iostream>
#include<string.h>
using namespace std;
int main()
{
string n = "Kebede";
cout<<n.at(0)<<endl;
cout<<n.size()<<endl;
cout<<n.at(n.size()-1)<<endl;
return 0;
}
```

Thank U all!!!!

Chapter 5

Functions

Functions

- A function is a collection of statements that perform a specific task.
- A c++ program is typically written by combining **new functions** with “prepackaged” functions available in C++ Standard Libraries.
- Most useful programs are much larger than the programs that we have considered so far. To make large programs manageable, we **modularize them into subprograms**.
- *Functions* allow the programmer to **modularize a program** by separating its tasks into self-contained units.
- Once a function is written, it can be *called* within other parts of the program.

C++ Library Functions

Function	Description
<code>sqrt(x)</code>	square root of x
<code>pow(x, y)</code>	x^y
<code>sin(x)</code>	sine of x (in radians)
<code>cos(x)</code>	cosine of x (in radians)
<code>tan(x)</code>	tangent of x (in radians)
<code>exp(x)</code>	e^x
<code>log(x)</code>	(natural log) $\ln(x)$, $x > 0$
<code>log10(x)</code>	(decimal log) $\lg(x)$, $x > 0$
<code>fabs(x)</code>	absolute value $ x $

Standard C++ Library Functions

- is a collection of **pre-defined functions** and other program elements which are accessed through *header files*.
 - Its actual code is hidden away within the Standard C++ Library.
 - Example :
 - the `sqrt()` function defined in `<cmath>`

User Defined Functions

The functions provided by the Standard C++ Library are still not sufficient for most programming tasks. Programmers also need to be able to define their own functions.

Syntax to create a function

```
type FunctionName( type Parameter1, type Parameter 2,...)
{
//Body of function goes here
}
```

Example: 1) int areaOfRectangle(int base, int height)

```
{
    int area=base*height;
    return area;
}
```

2) float cube(float x)

```
{
    float cube=x*x*x;
    return cube;
}
```

Example

```
#include<iostream>
```

```
int AddTwoNumbers();
```

← Function prototyping

```
using namespace std;
```

```
int main()
```

← Function calling

```
{
```

```
float x;
```

← Function definition

```
x=AddTwoNumbers();
```

```
cout<<"a+b="<<x;
```

```
return 0;
```

```
}
```

```
float AddTwoNumbers()
```

Function Prototypes

- Before the **compiler** encounters a call to a particular function, it **must already know** certain things about the function.
- In particular, it must know the **number of parameters the function uses**, the **type of each parameter**, and the **return type of the function**.
- One way of ensuring that the compiler has this required information is to place the function definition before all calls to that

Contd...

- E.g.
- `float AddTwoNumbers();`
- This prototype looks similar to the **function header**, except there is a **semicolon at the end**.
- Function prototypes are **usually placed near the top of a program** so the compiler will encounter them before any function calls.

Calling a Function

- A function is executed when it is *called*.
- Function **main** is **called automatically** when a program starts, but **all other** functions must be executed by *function call* statements.
- A *function call* is simply the name of the function followed by a set of arguments enclosed in parentheses and a semicolon.
- E.g.
`x=AddTwoNumbers();`

Function definition

- A function definition contains the statements that make up the function.
- All function definitions have the following parts:
- Return type ,Name, Parameter list and Body :

The diagram illustrates the components of a C++ function definition using the example: `int main () { cout << "Hello World\n"; return 0; }`. Arrows point from labels to the corresponding parts of the code: 'Return type' points to 'int', 'Name' points to 'main', 'Parameter list (This one is empty)' points to '()', and 'Body' points to the code block between the curly braces.

```
Return type      Name      Parameter list (This one is empty)      Body
   |               |               |               |
   v               v               v               v
int main () {
    cout << "Hello World\n";
    return 0;
}
```

Contd...

```
#include <iostream>
```

```
int add( int a, int b); // function declaration or prototyping
```

```
int main()
```

```
{
```

```
    int a,b;
```

```
    cout<< "Enter Two integers " <<endl;
```

```
    cin>> a >> b;
```

```
    cout<< "The sum of "<< a << " and " << b << " is " << add( a , b ); // function call
```

```
    return 0;
```

```
}
```

```
int add(int a , int b)  // function definition
```

```
{
```

```
    int sum;
```

```
    sum=( a + b ); // Replace function call with sum
```

```
    return sum;
```

```
}
```

N.B: If declaration is cancelled, function definition has to come before main function


```
#include<iostream>
```



prototype

```
using namespace std;
```

```
float AddTwoNumbers(float a, float b);
```

```
int main()
```

```
{
```

```
float n, m;
```

```
cout<<"Enter first number:";
```

```
cin>>n;
```

```
cout<<"Enter second number:";
```

```
cin>>m;
```

```
cout<<n<<"+"<<m<<"="<<AddTwoNumbers( n, m );
```

```
return 0;
```

```
}
```

```
float AddTwoNumbers(float a, float b)
```

```
{
```

```
return a+b;
```

```
}
```

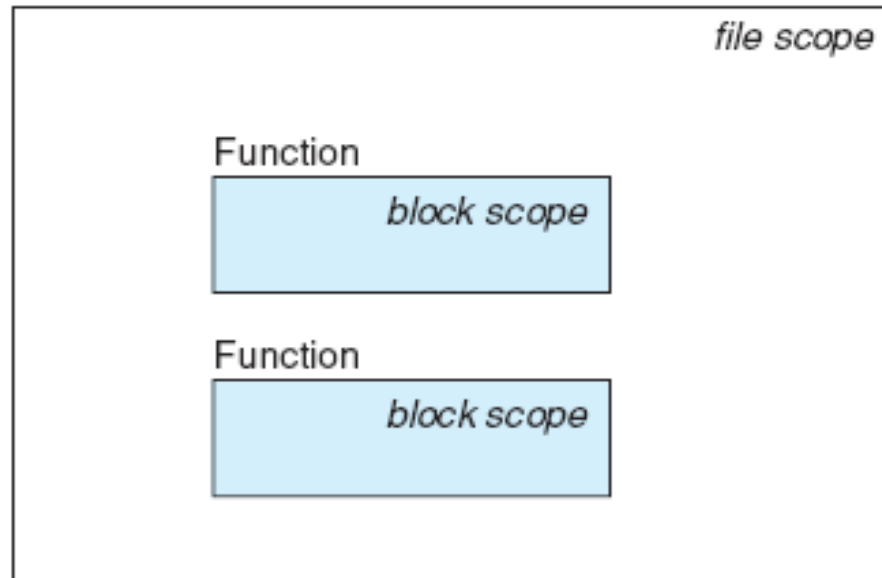
definition

Scope of variables

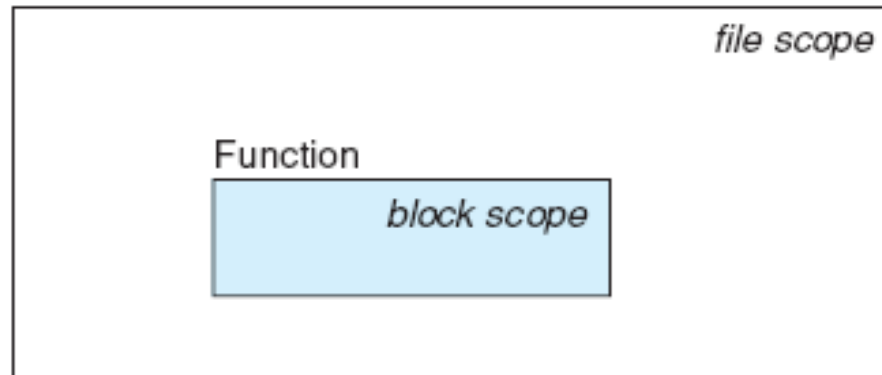
- **Scope**: is the **portion of a program** where an identifier or a variable name can be used
 - »Function scope
 - »File scope
 - »Block scope
 - »Function-prototype scope (program scope)

program scope

Module 1



Module 2



Example

```
#include<iostream>
using namespace std;
int main()
```

```
{
```

```
int x=0;
```

```
for(int b=0;b<5;b++)
```

```
{
```

```
int k=15;
```

```
x+=b;
```

```
cout<<b;
```

```
}
```

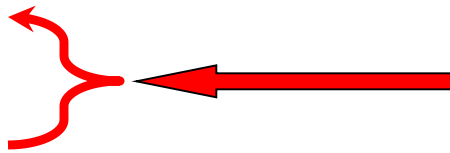
```
cout<<x;
```

```
cout<<b;
```

```
cout <<k;
```



**'b' & 'k' are accessible only
in this block**



**'b' & 'k' are not declared
in this scope**

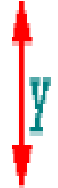
Argument passing By Value

We call function *addition* passing the values of x and y (5 and 3 respectively), not the variables themselves.

```
int addition (int a, int b)
z = addition (  5 ,  3 );
```

Argument passing By Reference

We call function *addition* passing the references of x and y (that means references of 5 and 3 respectively) ,not the values.

```
void duplicate (int& a,int& b,int& c)
  
duplicate ( x , y , z );
```

Example on Argument Passing by reference

```
//parameters passing by reference
#include<iostream>
int duplicate( int &a,int &b,int &c)
{
    a*=2;
    b*=2;
    c*=2;
}
int main()
{
    int x=1,y=3,z=7;
    duplicate(x,y,z);
    cout<<"x="<<x<<" "<<"y="<<y<<" "<<"z="<<z;
    return 0;
}
```

Example on Argument Passing by value

```
#include<iostream>
using namespace std;
int add(int number);
int main()
{
    int number,result;
    number=5;//enter a number
    cout << " The initial value of number is " << number << endl;
    result=add(number);
    cout << " The final value of number is " << number << endl;
    cout << " The result is : " << result << endl;
    return(0);
}
int add(int number)
{
    number=number+100;
    return(number);
}
```

```

#include<iostream>//example two by reference
using namespace std;
int add(int &number);
int main ()
{
    int number;
    int result;
    number=5;
    cout << "The value of the variable number before calling the function : " << number <<
endl;
    result=add(number);
    cout << "The value of the variable number after the function is returned : " << number <<
endl;
    cout << "The value of result : " << result << endl;
    return(0);
}
int add(int &p)
{
    p=p+100;
    return p;
}

```


Returning More than one value

```
//More than one returning value
#include<iostream.h>
void prevnext(int x,int &prev,int &next)
{
    prev=x-1;
    next=x+1;
}
int main()
{
    int x=100,y,z;
    prevnext(x,y,z);
    cout<<"Previous="<<y<<","Next="<<z;
    return 0;
}
//out previous=99,next=101
```

Function Overloading

Means defining functions having the same name but that take different parameters

```
#include<iostream.h>
```

```
int divide(int a,int b)
```

```
{  
return (a/b);  
}
```

```
float divide(float a,float b)
```

```
{  
return(a/b);  
}
```

```
int main()
```

```
{  
int x=5,y=2; float n=5.0,m=2.0;  
cout<<divide(x,y);  
cout<<"\n";  
cout<<divide(n,m);  
return 0;}
```

Function overloading

Are functions with the same name having:

- Different number of argument
- Different type of argument
- Both

//illustrates overloading the function name ave.

```
#include <iostream>
```

```
using namespace std;
```

```
double ave(double n1, double n2);
```

```
double ave(double n1, double n2, double n3);
```

```
int main( ) {
```

```
cout << "The average of 2.0, 2.5, and 3.0 is " << ave(2.0, 2.5, 3.0) << endl;
```

```
cout << "The average of 4.5 and 5.5 is " << ave(4.5, 5.5) << endl;
```

```
return 0;
```

```
}
```

```
double ave(double n1, double n2)
```



Two argument

```
{
```

```
    return ((n1 + n2)/2.0);
```

```
}
```

```
double ave(double n1, double n2, double n3)
```



Three Argument

```
{
```

```
    return ((n1 + n2 + n3)/3.0);
```

```
}
```

```
int addition (int a, int b)
```

```
{  
    int r;  
    r=a+b;  
    return (r);  
}
```

```
float addition (float a, float b)
```

```
{  
    float r;  
    r=a+b;  
    return (r);  
}
```

```
int main ()
```

```
{  
    int z; float y;  
    z = addition (5,3);  
    y = addition (5.5,3);  
    cout << "The result is " <<  
        z<<endl<<y;  
    return 0;  
}
```

```
int addition (int a, int b)
```

```
{  
    int r;  
    r=a+b;  
    return (r);  
}
```

```
int addition (int a, int b, int c)
```

```
{  
    int r;  
    r=a+b+c;  
    return (r);  
}
```

```
int main ()
```

```
{  
    int z; float y;  
    z = addition (5,3);  
    y = addition (5,5,3);  
    cout << "The result is " << z<<endl<<y;  
    return 0;  
}
```

Exercise

1. Suppose you have two function definitions with the following declarations:

`double score(double time, double distance);`

`int score(double points);`

Which function definition would be used in the following function call and why would it be the one used? (x is of type double.)

`double finalScore = score(x);`

2. Suppose you have two function definitions with the following declarations:

`double theAnswer(double data1, double data2);`

`double theAnswer(double time, int count);`

Which function definition would be used in the following function call and why would it be the one used? (x and y are of type double.)

`x = theAnswer(y, 6.0);`

Default values in arguments

```
#include<iostream.h>
```

```
int divide(int a, int b=2)
```

```
{
```

```
int r;
```

```
r=a/b;
```

```
return(r);
```

```
}
```

```
int main()
```

```
{
```

```
cout<<divide(12);//6
```

```
cout<<endl;
```

```
cout<<divide(20,4);//5
```

```
return 0;
```

```
}
```

```
//out put 6 and 5
```

N.B

- Though the 2nd parameter is not specified when calling function “divide” it takes the 2nd value of divide function that is found @ function prototyping i.e. the default value in argument.

```
include <iostream>

Using namespace std;

void showVolume(int length, int width = 1, int height = 1);

int main( )
{
showVolume(4, 6, 2);
  showVolume(4, 6);
showVolume(4);

return 0;
}

void showVolume(int length, int width, int height)
{
cout << "Volume of a box with \n"
<< "Length = " << length << ", Width = " << width << endl<<
"and Height = " << height<<
" is " << length*width*height << endl;
}
```

Recursive functions:

- Is Function that calls themselves.

Example 6

```
long factorial(long n)
{
    if(n==0)return 1;
    else
return n*factorial(n-1);
}
```

Or

```
int Factorial (unsigned int n)
{
return n == 0 ? 1: n * Factorial (n-1);
}
```



```
Void main()
{
int n;
cout<<"enter a +ve number:";
cin>>n;
cout<<"the factorial of
    "<<n<<"is"<<factorial(n);
}
```

Recursively

- //factorial calculator

```
#include<iostream.h>

long factorial (long a)
{
    if(a>1)
        return (a* factorial(a-1));
    else
        return 1;
}

int main()
{
    long l;
    cout<<"Type a number:";
    cin>>l;
    cout<<"!"<<l<<"="<<factorial(l);
    return 0;
}
```

Inline Function

- If a function is inline, the compiler places a copy of the code of that function at each point where the function is called at compile time.
- To inline a function, place the keyword **inline** before the function name and define the function before any calls are made to the function.

Passing array as argument to a function

- We pass both array and its size to a function.
- Array are passed as reference variable without (&)

Example.

```
double read(int a[],int size);
```

- When we **call function** with array parameter we simply pass array name and size.

```
read(a,size);
```

```

// the use of function prototype
#include<iostream.h>
int area(int length,int width);
int main()
{
int l;
int w;
int areaofrect;
cout<<"enter lenth of the rectangle:";
cin>>l;
cout<<"Enter width of the rectangle:";
cin>>w;
areaofrect=area(2,10);
cout<<"Area of the rectangle:"<<areaofrect<<endl;
return 0;
}
int area( int l,int w)
{
return l*w;
}

```

```

#include<iostream.h>
int Volume(int length,int width=5,int height=1);
int main()
{
int length=2,width=2,height=3;
int volume;
volume = Volume(length, width, height);
cout<<"First area equals:"<<volume<<"\n";//12
volume = Volume(length, width);
cout<<"second area equals:"<<volume<<"\n";//4
volume= Volume(length);//
cout<<"Third area equals:"<<volume<<"\n";
return 0;
}
Int Volume(int length, int width, int height)
{
return(length * width * height);
}

```

Chapter-6:Pointers

- The memory of your computer can be imagined as a **succession of memory cells**, each one of the minimal size that computers manage (one byte).
- These single-byte memory cells are numbered in a **consecutive way**, so as, within any block of memory, every cell has the same number as the previous one plus one.

- This way, each cell can be easily located in the memory because it has a unique address and all the memory cells follow a successive pattern.
- For example, if we are looking for cell 1776 we know that it is going to be right between cells 1775 and 1777
- It is exactly one thousand cells after 1776 and exactly one thousand cells before cell 2776.

Reference operator (&)

- The address that locates a variable within memory is a *reference to that variable*.
- reference to a variable can be obtained by preceding the identifier of a variable with an ampersand sign (&), known as *reference operator*, and which can be literally translated as "*address of*".
- For example:
- `Int var=20;`

`Int *pointer;//pointer=&var;`

`int pointer = &var; //Ptr now holds the address`

Example:

```
#include<iostream.h>
```

```
int main()
```

```
{
```

```
int var1=10,var2=20,var3=30;
```

```
cout<<endl<<&var<<endl<<&var2<<endl<<&var3;
```

```
Return 0;
```

```
}
```

- The **variable** that stores the reference to **another variable** is called a *pointer*.
- In other words, **pointers** are variables that hold **an address value**.

pointers :

- offering a very efficient method of accessing data
- they provide efficient technique for manipulating data in arrays
- they are used in functions for reference parameters
- for dynamic allocation of memory.

Dereference operator (*)

- Pointers are said to "**point to**" the variable whose reference they store.
- Using a pointer we can **access the value stored in the variable** which it points to.
- `Value = *Ptr;/////` read as: "Value equals to value pointed by Ptr"
- `Ptr` refers to the address location(say 1776)
- `*Ptr` refers to the value stored at address 1776.

Structures&Enumerations

- In many occasions what we want to store are not mere sequences of elements all of the same data type(as in arrays), but sets of different elements with different data types.
- Structures offer a way of joining these heterogeneous elements together to form complex structures.

- A structure is **a group of data elements** grouped together under one name. These data elements, known as **members**, can have *different* types and different lengths. Structures are declared in C++ using the following syntax:

```
struct structure_name //struct is a keyword
{
member_type1 member_name1;
member_type2 member_name2;
member_type3 member_name3;
} object_names;
```

- structure_name is a name for the structure type,

- **object_name** can be a set of valid identifiers for objects that have the type of this structure.
- Within **braces { }** there is a **list with the data members**, each one is specified **with a type and a valid identifier as its name**.
- **Example:**

```
struct person {  
    float height;  
    int eye_color;  
    char name[20]  
};  
person mother; person friend, spouse;
```

- We can also declare the structure objects *mother*, *father* and *spouse* at the moment we define the structure like this way:

```
struct stud1 {  
    float height;  
    int eye_color;  
    char name[20]  
} mother, friend, spouse;
```


friend

name

eye_colour

height

mother

name

eye_colour

height

spouse

name

eye_colour

height

Accessing structure Members

- We use the *dot operator (.)* to access members of a structure. We use this format to read, to write or assign data to members of a structure. We can assign value to the name of stud1 as follows:
 - `stud1.name = "Abebe";`
 - `stud1.eye_color=1;`
 - `Stud1.height=1.70;`
 - Generally, structure members are treated just like other variables.

friend

name
Diane
eye_colour
1
height
1.61

mother

name
Mary
eye_colour
2
height
1.44

spouse

name
Helen
eye_colour
1
height
1.7

Exercise

1. Define two functions which, respectively, input values for the elements of an array of reals and output the array elements:
 - `void ReadArray (double nums[], int size);`
 - `void WriteArray (double nums[], int size);`

Exercise on functions

1. What are the differences between the function prototype and the function definition?

The function prototype declares the function; the definition defines it. The prototype

- ends with a semicolon; the definition need not.
- The declaration need not include names for the parameters; the definition must.

2. Do the names of parameters have to agree in the prototype, definition, and call to the function?

No All parameters are identified by position, not name.

3. If a function doesn't return a value, how do you declare the function?

Return to void

4. If you don't declare a return value, what type of return value is assumed?

By default returns int

5. What is a local variable? A local variable is a variable passed into or declared within a block,
6. What is scope? Scope refers to the visibility and lifetime of local and global variables. Scope is usually established by a set of braces.
7. What is recursion? generally refers to the ability of a function to call itself.
8. When should you use global variables?
typically used when many functions need access to the same typically used when many functions need access to the same data.

Exercise on Array

1. What are the first and last elements in `SomeArray[25]`?

2. How do you declare a multidimensional array?

eg `int theArray[5][3];`

3. Initialize the members of an array declared as `SomeArray[2][3][2]`.

For example, `SomeArray[2][3][2]` is

a three-dimensional array. The first dimension has two elements, the second has three, and the third has two.

3. `SomeArray[2][3][2] = { { {1,2},{3,4},{5,6} }, {7,8},{9,10},{11,12}}};`

4. How many elements are in the array `SomeArray[10][5][20]`?

5. How does a linked list differ from an array?

6. How many characters are stored in the string “Jesse knows C++”?

16 characters = 13 letters+2 space+one null char

7. What is the last character in the string “Brad is a nice guy”? Null character

Exercise on loops

What is the value of x when the for loop completes?

```
for (int x = 0; x < 100; x++) //out of scope it has no valid value
```

1) Write a nested for loop that prints a 10×10 pattern of 0s.

```
for (int i = 0; i < 10; i++)
```

```
{
```

```
for ( int j = 0; j < 10; j++)
```

```
cout << "0";
```

```
cout << endl;
```

```
}
```

2) Write a for statement to count from 100 to 200 by twos.

```
for (int x = 100; x <= 200; x += 2)
```

3) .Write a while loop to count from 100 to 200 by twos.

```
int x = 100;
```

```
while (x <= 200)
```

```
x += 2;
```


4. Write a do while statement to count from 100 to 200 by twos.

```
int x = 100;  
do  
{  
x+=2;  
} while (x <= 200);
```

4. Write a do...while loop to count from 100 to 200 by twos.

```
int x = 100;
```

```
do
```

```
{
```

```
x+=2;
```

```
} while (x <= 200);
```